hashlock.

Security Audit

Swoobz (GameFi)

June 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

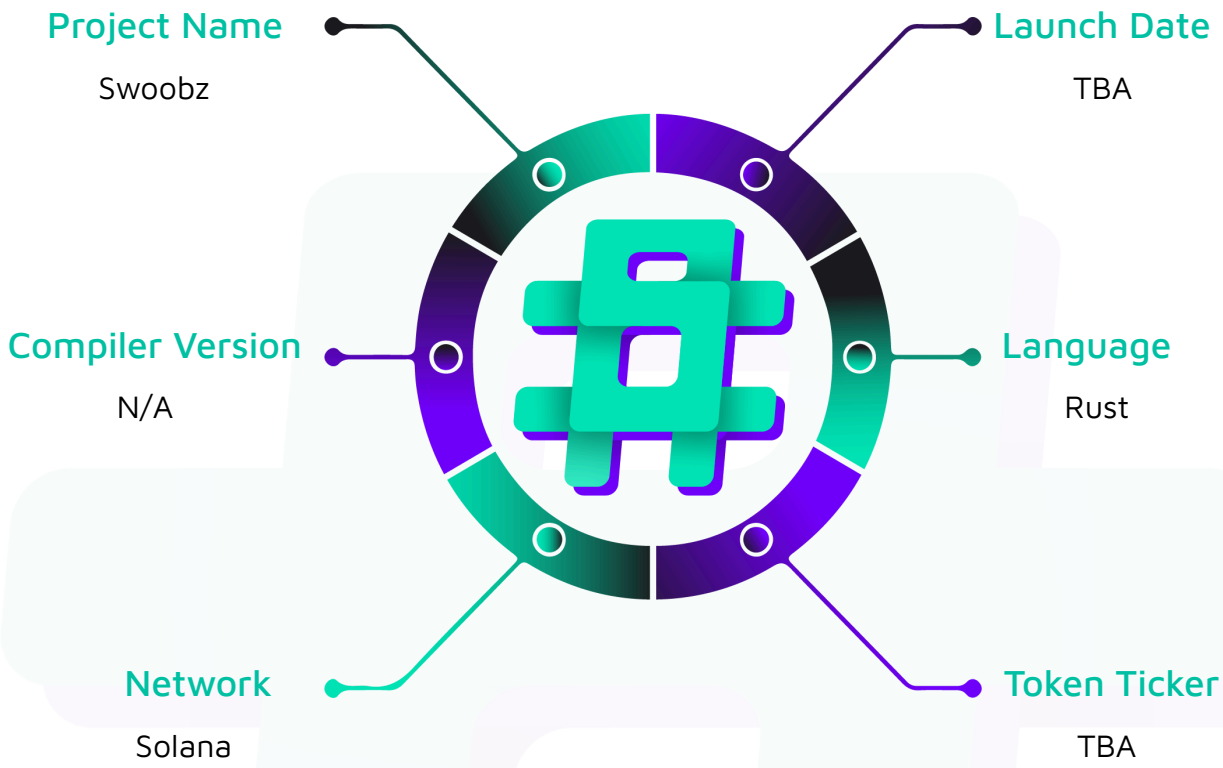
The Swoobz team partnered with Hashlock to conduct a security audit of their code. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

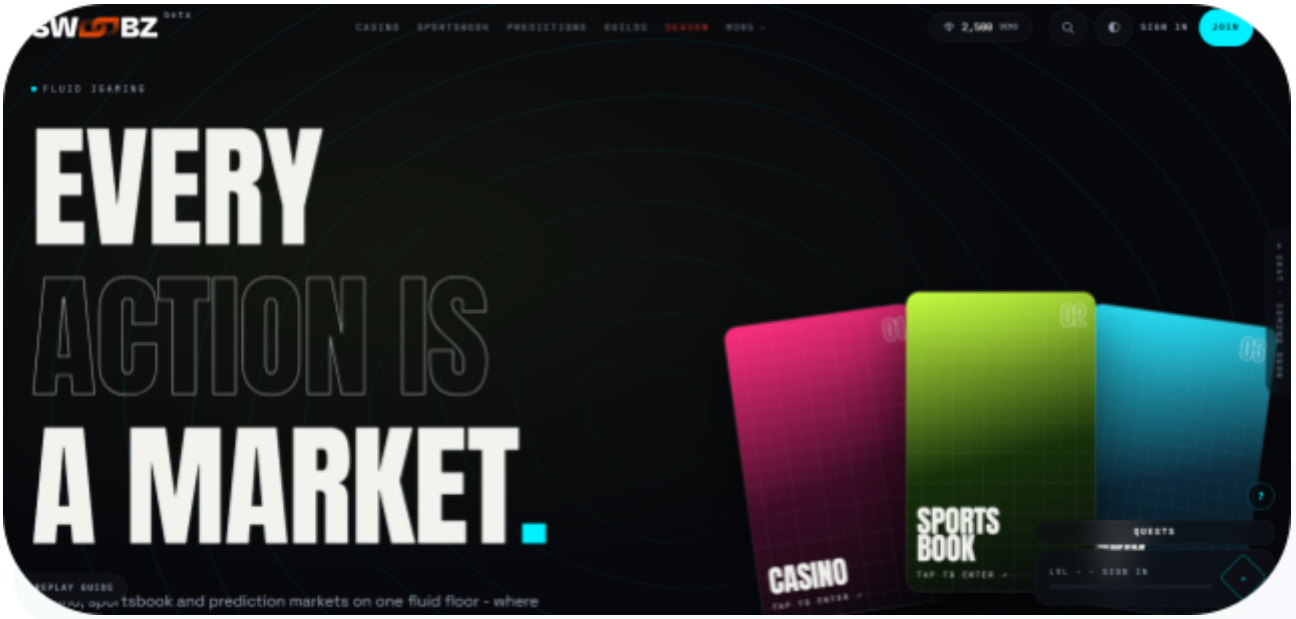
Swoobz is a Web3 iGaming platform. It combines blockchain technology with online gaming, giving users access to decentralized casino games, a personalized sportsbook, predictions, and digital asset-based rewards. The platform offers a transparent and engaging gaming experience through crypto payments and blockchain infrastructure. At the same time, it aims to modernize traditional online gambling with Web3 technologies, personalized betting tools, and user-owned digital assets.

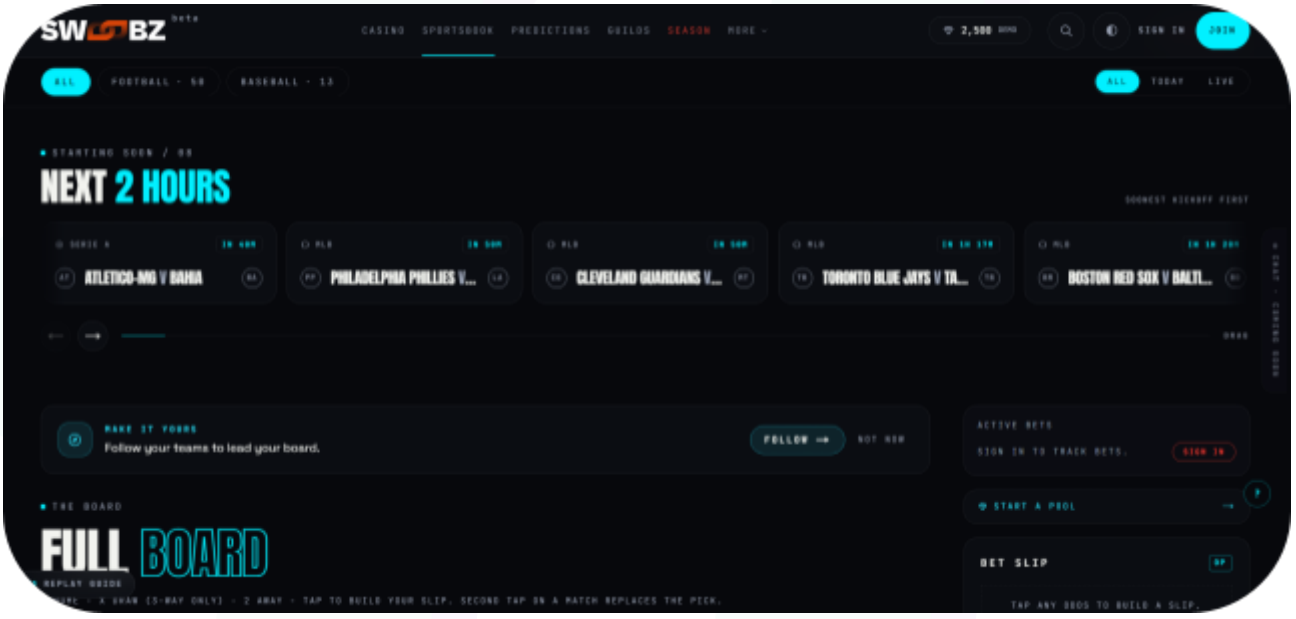
Project Name: Swoobz
Project Type: GameFi
Website: https://swoobz.com/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Rust code within the Swoobz project, the scope of work included a comprehensive review of the programs and modules listed below. We tested the programs to assess their security, functionality, and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Swoobz Smart Contracts**
Platform	**Solana / Rust**
Audit Date	**July, 2026**
Contract 1	swoobz-token
Contract 2	house-pool
Contract 3	revenue-distributor
Contract 4	staking-veswoobz
Contract 5	buyback-burn
Contract 6	transfer-hook
Contract 7	game-router-core
Contract 8	game-router-ref
Audited GitHub Commit Hash	dee47021fbf40dbb7e47dc8ecfe40c03464d61ac
Fix Review GitHub Commit Hash	32fcc817c09e6c2b813d8615bbce56aa18b6d7cd

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the codebase to be **"Secure"**. The code all follows simple logic, with correct and detailed ordering. They use a series of interfaces and external dependencies, including standard open-source libraries.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Program Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

7 High severity vulnerabilities

15 Medium severity vulnerabilities

14 Low severity vulnerabilities

10 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Program Functions

Claimed Behaviour	Actual Behaviour
programs/game-router-core provides: - Canonical settlement and player-authorization digest generation. - Ed25519 verification for player authorizations and oracle attestations. - CommitReveal seed verification and deterministic binary payout derivation. - State-account layouts, PDA seeds, and PDA derivation helpers. - Typed, PDA-signed CPI wrappers for the house-pool wager and payout operations. - CommitReveal and Ed25519-oracle settlement primitives.	**The library is delivered as a reusable Rust/Anchor crate rather than a standalone deployed Solana program, and it is embedded and exercised end to end by game-router-ref. The settlement primitives it exposes do not hold in their current form. The committed server seed is not bound to player consent, the commit-reveal verifier permits outcome-dependent reveal aborts, and oracle keys can neither be rotated nor revoked.**
programs/swoobz-token provides: - Atomic-genesis deployment of the canonical $SWOOBZ Token-2022 mint with a fixed four-extension set (TransferFeeConfig, TransferHook, MetadataPointer, TokenMetadata). - Full-supply mint-to-treasury and same-instruction mint-authority revocation. - Two-phase timelocked fee-update governance (propose then cancel or execute) bounded by a	**The claimed functionality is achieved, with one QA-tier note on metadata-update authority immutability being enforced by handler-absence rather than declaratively at the Token-2022 layer.**

2000-bps ceiling. - Anti-bot-fee reducer. - Treasury-pinned withdraw_fees CPI for harvesting withheld transfer fees.	
programs/house-pool provides: - Custody of USDC bankroll in retail and institutional tranches. - LP-share mint and burn against pool NAV. - Typed CPI surface for authorized game programs to collect wagers and reserve or release payouts. - Per-game and per-player exposure tracking with Kelly-sized bet capacity. - Retail-withdrawal velocity throttles and loss-based circuit breakers. - Insurance-draw and recapitalization surfaces gated by a five-class authority matrix (governance, operational, guardian, treasury, compliance).	**The core custody, tranche, LP-share, and Kelly-cap functionality is achieved, with multiple defense-in-depth gaps documented in the audit findings.**
programs/revenue-distributor provides: - Accumulation of USDC revenue from authorized casino / BetBy source programs and Token-2022 SWOOBZ transfer-fee harvests. - Per-epoch distribution of net profit across six named buckets (burn, player rewards, LP yield, PoW service, insurance, ops / growth) by governance-configured basis-point ratios. - Emergency-buyback redirect gated by a dual timelock (wall-clock + slot) with executor-vs-proposer separation of duties. - Insurance-cap sizing off an authority-maintained recorded balance to prevent donation-based inflation.	**The claimed functionality is achieved and no findings were surfaced during multi-methodology review.**
programs/staking-veswoobz provides:	**The intended lock,**

- Locks $SWOOBZ for a chosen duration (30, 90, 180, or 365 days) and computes vote-escrow voting power = amount * multiplier. - Custody of staked SWOOBZ in a self-owned vault PDA. - Per-position lock lifecycle (create, extend, close). - Governance-participation records via VoteRecord PDAs. - Keccak256 merkle-root distribution of GPR and LP-yield rewards gated by a two-phase publish/activate timelock. - Permissioned work-record + merkle-claim rail for PoW-service fees.	**voting-power, and merkle-claim functionality is achieved, but two initialization-time issues prevent the program from booting against the production SWOOBZ mint until fixed.**
programs/transfer-hook provides: - Executes on every $SWOOBZ Token-2022 transfer to enforce the platform's compliance rules (sanctions, KYC-frozen, KYC-tier) via HolderStatusV1 PDAs on both the source and destination legs. - Global PauseState singleton and canonical-mint bind on every transfer. - Governance flows (propose / execute / cancel pause and rotation of the bound Squads multisig) with slot-based timelocks. - Sanctioned / frozen-flag mutations gated behind the sanctions-multisig authority. - ExtraAccountMetaList resolver and a per-wallet holder-status initialization surface.	**The hot-path enforcement and governance functionality is achieved, with one MEDIUM sanctions-evasion gap remaining because the sanctions flag does not couple to Token-2022 FreezeAccount..**
buyback-burn: Custody of the USDC buyback budget and the SWOOBZ accumulation vault under program-controlled PDAs. - Time-averaged buyback orders that acquire $SWOOBZ in metered slices, each bounded by a	**The claimed functionality is not achieved against the production $SWOOBZ mint. The SWOOBZ deposit leg of execute_twap_slice omits the Token-2022**

per-order worst-acceptable price set in a transaction distinct from any slice deposit. - Separation of the price-setting authority (Squads multisig) from the operational cranker authority. - An active price band with a slot-timelocked update path, into which every order's acquisition price is hard-clamped. - Measured-deposit slice accounting that credits the observed vault increase rather than the cranker's reported figure. - Permanent burn of accumulated $SWOOBZ with an immutable BurnRecord per burn, tagged by source.	**transfer-hook accounts, so the instruction reverts atomically and no slice can settle, stalling the buyback and burn cycle entirely. The price-band, authority-separation and measured-deposit defenses are implemented as described, with per-slice slippage read live rather than pinned at order creation, and cranker token accounts relying on implicit rather than declarative mint constraints.**
Game-router-ref: programs/game-router-ref provides: - A reference single-player bet lifecycle wired end to end on top of the game-router-core library. - Wager collection into a per-bet escrow and maximum-payout reservation in the bankroll program. - On-chain outcome settlement, proved before any funds move, written to a verified outcome record. - Win payout and loss sweep through the bankroll CPI surface. - A 1:1 refund path for rounds that never settle, so player funds cannot be stranded.	**The lifecycle sequencing is implemented as described, but several of the settlement guarantees it depends on do not hold in their current form. Player consent is not bound to the committed server seed, the commit-reveal flow permits outcome-dependent reveal aborts, and a game-wide nonce allows permissionless admission denial of service, alongside replayable cross-deployment authorizations, signed wagers that neither expire nor can be cancelled, and unvalidated max_payout bounds at admission.**

Code Quality

This audit scope involves the smart contracts of the Swoobz project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Swoobz project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] house-pool::void_bet / release_payout / release_reserved / collect_bet - Permanent player USDC lock when the game program cannot CPI-sign - no admin recovery path exists at any authority tier

Description

Every money-path handler that touches a Reservation or `bet_escrow` (`collect_bet`, `reserve_payout`, `release_payout`, `release_reserved`, `void_bet`) requires the `game_authority` PDA of the specific game program to sign the CPI. `game_authority` is derived deterministically as `PDA([b"game_exposure", game_program_id], house_pool_program_id)`, so only the game program itself, via `invoke_signed`, can produce a valid signature - not the game team's off-chain wallet, not the program's upgrade authority, not any operator key.

If the game program becomes permanently unable to CPI (a bug on the settle path, upgrade authority revoked before a bug is discovered, program upgraded to malicious code that selectively refuses to void, off-chain settlement engine shut down, project abandoned, or the pubkey registered as `game_program` was never executable to begin with because `authorize_game.rs` accepts any `UncheckedAccount` as `game_program` with no `executable` check) then every open Reservation for that game is permanently unsettleable. Player USDC stays trapped in `bet_escrow` PDAs, `player_balance.locked_in_play` is inflated forever, `player_withdraw` cannot access those funds, and there is no on-chain path at any authority tier - operational, treasury, guardian, governance, or compliance - to recover them. The `void_bet.rs` header comment states its purpose is *"a player's wager must never be permanently stranded inside `bet_escrow`"* - the current implementation does not achieve this stated invariant.

A deauthorization workaround is also blocked: `deauthorize_game` requires

`game_exposure.current_exposure == 0`, but reducing `current_exposure` requires one of the three settle paths, which require the (dead) game to sign. The result is a permanent deadlock.

Vulnerability Details

`void_bet`, `release_payout`, `release_reserved`, and `collect_bet` all require the `game_authority` PDA of the specific game program to sign the CPI AND require `game_exposure.authorized == true`. `game_authority` is derived as `PDA([b"game_exposure", game_program_id], house_pool_program_id)` (`game-router-core/src/cpi/builders.rs:217-230`), so only the game program itself, via `invoke_signed`, can produce a valid signature - no keypair, no upgrade authority, and no operator key can substitute. The gate applied on every settle path is shown below:

```
// programs/house-pool/src/instructions/void_bet.rs:52, 65-76
#[account(mut)]
pub game_authority: Signer<'info>,

#[account(
    mut,
    seeds = [GAME_EXPOSURE_SEED, game_authority.key().as_ref()],
    bump = game_exposure.bump,
    constraint = game_exposure.authorized
        @ HousePoolError::GameNotAuthorized,
    constraint = game_exposure.game_program == game_authority.key()
        @ HousePoolError::GameProgramMismatch,
)]
pub game_exposure: Box<Account<'info, GameExposure>>,
```

`deauthorize_game` cannot unstick the deadlock either - it requires `game_exposure.current_exposure == 0`, and `current_exposure` decrements only via the three settle paths that need the (dead) game to sign. `player_withdraw` cannot rescue the stake because it operates on `PlayerBalance.balance`, which by construction excludes `locked_in_play`. No handler in `lib.rs` (`admin_void_bet`, `force_settle`, `guardian_void`, etc.) exists at any authority tier - operational, treasury, guardian, governance, or compliance - that can decrement `current_exposure`, drain `bet_escrow`, close a Reservation, or reduce `locked_in_play` without the game's signature. The trap is

amplified by the fact that `authorize_game.rs` accepts any `UncheckedAccount` as `game_program` with no `executable` check, so a single operational typo creates a permanent trap from the first bet placed under that registration.

Proof of Concept

Attack / failure scenario (non-adversarial trigger - game program failure):

1. Governance authorizes Game A via `authorize_game`. `game_exposure.authorized = true`, `game_exposure.game_program = A`. Rent paid by operational authority.
2. 500 players place bets over the following week. Each `collect_bet` succeeds; each moves `bet_amount` USDC from `player_vault → bet_escrow` and increments `player_balance.locked_in_play`. Each `reserve_payout` bumps `pool_config.total_exposure`, `game_exposure.current_exposure`, and `player_exposure.current_exposure`.
3. At this point 500 `Reservation` PDAs exist, 500 `bet_escrow` PDAs hold player USDC, and the pool's exposure counters carry the aggregate.
4. Game A ships an upgrade with a bug that reverts on the settle-path CPI. Alternatively: Game A's off-chain VRF operator is shut down; or Game A's team abandons the project and no one triggers CPIs anymore; or Game A's upgrade authority was revoked before the bug was discovered.
5. Governance attempts `deauthorize_game(A)` - fails with `HousePoolError::GameHasActiveExposure` because `current_exposure > 0`.
6. Players wait past `BET_VOID_TIMEOUT_SECONDS = 120s`. Players themselves cannot call `void_bet` - the handler requires `game_authority: Signer<'info>` and only Game A's on-chain code can produce that signature.
7. Governance attempts to recover via emergency pause, guardian intervention, treasury draw, parameter update - no handler exists that decrements `current_exposure` or `locked_in_play` without Game A's signature.
8. Result: 500 × `bet_amount` USDC is permanently locked in `bet_escrow` PDAs. 500 × `bet_amount` is added to `player_balance.locked_in_play` counters that will never decrement. `player_withdraw` cannot access this money because `balance` excludes `locked_in_play`. `pool_config.total_exposure` is inflated forever, tightening the Kelly cap for the rest of the pool's operational life.

Registration-error variant (Day-1 trap via missing `executable` constraint on `authorize_game`):

1. Operational multisig executes `authorize_game(game_program = 0xTYP0...)` - a typo pubkey that is not an executable program.
2. `authorize_game` succeeds (no `executable` check). `game_exposure` is created and `authorized = true`.
3. First `collect_bet` under this "game" from a player: succeeds structurally (the game_authority PDA is derivable even for non-programs, and the `Signer<'info>` constraint is checked at CPI time). But since no program exists at the seed pubkey, no CPI can ever produce a valid `invoke_signed` for the settle path.
4. Every bet placed under this misregistered game is stuck forever, from the first one.

Impact

Permanent USDC principal loss for players (not rent): full aggregate `Σ(bet_amount)` across every open Reservation for a dead game is trapped in `bet_escrow` forever, with `player_balance.locked_in_play` inflated indefinitely. No on-chain recovery exists at any authority tier, and `pool_config.total_exposure` stays inflated forever, tightening the Kelly cap on all future bets. Realistic triggers include game program bugs, revoked upgrade authority, off-chain oracle shutdown, and project abandonment; the missing `executable` constraint on `authorize_game` compounds into a Day-1 permanent trap on a single registration typo.

Recommendation

Add a `guardian_authority`-gated `admin_void_bet(game_program, player, bet_nonce)` handler that mirrors `void_bet.rs:200-297`'s refund + close logic but drops the `game_authority` signer and the `authorized == true` gate, keyed by seed instead of signer. Gate it behind a long grace window (e.g. `ADMIN_VOID_GRACE_SECONDS = 30 * 86_400`, 15,000× the fast-path `BET_VOID_TIMEOUT_SECONDS`) so operators exhaust legitimate settlement first, and route Reservation rent to `pool_authority` instead of the dead game. Additionally add a `constraint = game_program.executable` clause on

`authorize_game`'s `game_program: UncheckedAccount<'info>` to eliminate the registration-typo variant.

Status

Resolved

[H-02] staking-veswoobz::initialize_vaults - Hardcoded 165-byte Token-2022 vault account size incompatible with SWOOBZ mint extensions produces permanent bootstrap failure

Description

`initialize_vaults` creates three self-owned Token-2022 vault accounts (`staking_vault`, `gpr_reserve_vault`, `pow_fee_vault`) with a hardcoded `account_size: u64 = 165` - the base Token-2022 `Account` layout with zero extensions. The SWOOBZ mint carries four extensions (`TransferFeeConfig`, `TransferHook`, `MetadataPointer`, `TokenMetadata`; see `swoobz-token/src/instructions/initialize_mint.rs:22-27`). Of those, `TransferFeeConfig` and `TransferHook` on the mint side REQUIRE the corresponding account-side extensions (`TransferFeeAmount` and `TransferHookAccount`) on every account of that mint. Token-2022's `initialize_account3` calls `ExtensionType::get_required_init_account_extensions` on the mint, computes the required account length, and returns `InvalidAccountData` if the account is smaller than that length. A 165-byte allocation is insufficient - all three `create_and_init_vault` CPIs will fail atomically, blocking the entire staking-veswoobz bootstrap. The correct pattern already exists in the same codebase at `swoobz-token/initialize_mint.rs:358` (`ExtensionType::try_calculate_account_len::<Token2022Mint>(&[...])`), but is not reused here.

Vulnerability Details

`programs/staking-veswoobz/src/instructions/initialize_vaults.rs:90-91` - hardcoded size:

```
// programs/staking-veswoobz/src/instructions/initialize_vaults.rs:90-91
// Token account size for Token-2022
let account_size: u64 = 165;
```

`programs/staking-veswoobz/src/instructions/initialize_vaults.rs:189-198` - the `initialize_account3` CPI that will reject the under-sized account:

```
// programs/staking-veswoobz/src/instructions/initialize_vaults.rs:189-198
invoke_signed(
    &spl_token_2022::instruction::initialize_account3(
```

```
        &spl_token_2022::ID,
        vault.key,
        mint_key,
        vault.key, // self-owned vault
    )?,
    &[vault.clone(), mint_info.clone(), token_program.clone()],
    &[signer_seeds],
)?;
```

`programs/swoobz-token/src/instructions/initialize_mint.rs:22-27` - the four mint-side extensions confirming SWOOBZ is a "requires account-side extensions" mint:

```
// programs/swoobz-token/src/instructions/initialize_mint.rs:22-27
const EXPECTED_MINT_EXTENSIONS: [ExtensionType; 4] = [
    ExtensionType::TransferFeeConfig,
    ExtensionType::TransferHook,
    ExtensionType::MetadataPointer,
    ExtensionType::TokenMetadata,
];
```

`programs/swoobz-token/src/instructions/initialize_mint.rs:358-363` - the correct sizing pattern already used in the sibling program:

```
// programs/swoobz-token/src/instructions/initialize_mint.rs:358-363
let fixed_len = ExtensionType::try_calculate_account_len::<Token2022Mint>(&[
    ExtensionType::TransferFeeConfig,
    ExtensionType::TransferHook,
    ExtensionType::MetadataPointer,
])
.map_err(|_| error!(SwoobzTokenError::InsufficientMintSpace))?;
```

Impact

`initialize_vaults` reverts atomically on the first `initialize_account3` CPI, permanently blocking staking bootstrap against the production SWOOBZ mint. Every downstream instruction (`create_position`, `close_position`, `claim`, `harvest`, `claim_epoch_reward`) is unreachable until the program is patched and re-deployed. Testing did not surface this because the fail-closed dev default of `SWOOBZ_MINT_PUBKEY = [0u8; 32]` never exercises the real extension-mint code path.

Recommendation

Replace the hardcoded `165` with a dynamic computation using Token-2022's own sizing helper:

```
use spl_token_2022::extension::ExtensionType;
use spl_token_2022::state::Account as Token2022Account;
let account_size = ExtensionType::try_calculate_account_len::<Token2022Account>(&[
    ExtensionType::TransferFeeAmount,
    ExtensionType::TransferHookAccount,
]).map_err(|_| error!(StakingError::InsufficientVaultSpace))? as u64;
```

Then thread this length into both the `create_account` rent computation and the CPI. Add a unit or integration test that binds `SWOOBZ_MINT_PUBKEY` to a fixture mint with the same 4 extensions and runs `initialize_vaults` end-to-end to prevent regression.

Status

Resolved

[H-03] game-router-core::compute_player_auth_digest - Player consent does not bind the committed server seed

Description

Player authorization can be rebound to a server seed selected after the player's entropy is known. The signed digest contains `game_id`, `round_or_bet_id`, player, wager, `max_payout`, `client_seed`, and nonce, but it does not contain the live `server_seed_hash` or commitment epoch in `programs/game-router-core/src/settlement/digest.rs:375-413`. Consequently, changing the live commitment does not invalidate an authorization that the player produced against the same nonce.

Even if the player reads a valid pre-existing server-seed commitment before signing, the authorization does not identify that commitment. After receiving the authorization, the administrator can reveal the old seed and install a replacement commitment while preserving the authorization's nonce. The old authorization is then accepted against the replacement hash and epoch. The code guarantees that a commitment exists before `place_bet`, but does not bind the off-chain signing event to that commitment.

Vulnerability Details

The intended provable-fairness invariant requires the house seed to be locked before the player chooses `client_seed`, specifically so the seed-controlling operator cannot grind against known player entropy in `programs/game-router-core/src/state/server_seed_commitment.rs:1-11`. Relayed submission is a supported standard flow and must remain safe. When a player uses a game-operated relayer, the relayer receives `client_seed` and the corresponding Ed25519 authorization before `place_bet` executes. The program accepts any signer as `fee_payer` in `programs/game-router-ref/src/lib.rs:963-995`, so the game administrator can operate or coordinate such a relayer and delay submission without compromising the player.

An administrator needs the reusable authorization and its signed `place_bet` parameters, including `client_seed`, rather than a complete outer transaction. These can be obtained

through the game's relayer, a colluding relay/RPC/validator path, or a recorded failed transaction.

After receiving an authorization, the administrator can grind candidate server seeds until one produces the desired outcome, then call rotate_server_seed. Rotation requires the game administrator, reveals the old seed, installs an arbitrary new commitment hash, increments epoch, and deliberately leaves `next_nonce` unchanged in `programs/game-router-ref/src/lib.rs:175-230`. The held authorization therefore remains valid.

Keeping `next_nonce` globally monotonic across epochs is sound by itself and prevents nonce reuse. The unsafe property is specifically that a commitment-changing rotation leaves the current pending authorization valid. Already-open rounds are not affected by how rotation advances `next_nonce`, because each RefRound snapshots its own nonce, commitment hash, and epoch before settlement.

When the relayer finally submits the bet, `place_bet` reads the replacement `server_seed_hash` and epoch while consuming the unchanged nonce, but verifies the player's signature over only the original narrow digest in `programs/game-router-ref/src/lib.rs:284-324`. It then persists the replacement hash and epoch into the round in `programs/game-router-ref/src/lib.rs:398-410`. A later rotation reveals the selected seed, and settlement validates it against the replacement commitment and re-derives the administrator-selected result as a legitimate outcome in `programs/game-router-ref/src/lib.rs:483-565`.

For the core commit-reveal path, this manipulated result is binary rather than a partial loss: `derive_commit_reveal_outcome` returns either the full reserved payout or exactly zero in `programs/game-router-core/src/settlement/digest.rs:365-372`, and `verify_commit_reveal` requires the claimed payout to match that value exactly in `programs/game-router-core/src/settlement/commit_reveal.rs:74-89`. Grinding a loss therefore fixes the player's payout at zero.

Impact and likelihood are both High. Players may legitimately use the game's relayer as

a standard submission path, in which case the administrator can obtain every required input directly; several alternative pre-execution observation paths are also available. Exploitation has low complexity and requires neither player compromise nor a cryptographic break. The administrator is semi-trusted for seed publication and rotation, but commit-reveal is expressly intended to prevent that role from adapting a seed to known player entropy; therefore the trusted control-plane cap does not apply and the resulting severity is Critical.

Proof of Concept

Starting from a commitment with current seed `seed_0`, epoch E, and `next_nonce` = N, use the normal player and administrator keys:

```
client_seed = player_random_seed()
auth = player.sign(player_auth_digest(game, round, wager, max_payout,
                                      client_seed, N))
player_claim_before = player_balance.balance + player_balance.locked_in_play
pool_ggr_before = game_exposure.total_collected


# The administrator's relayer holds the reusable player authorization and
# signed place_bet parameters instead of submitting them.
seed_1 = grind_until(
    derive_commit_reveal_outcome(candidate, client_seed, round, N,
                                 wager, max_payout, house_edge) == 0
)


rotate_server_seed(prior_seed = seed_0, new_hash = sha256(seed_1))
assert commitment.epoch == E + 1
assert commitment.next_nonce == N


# The unchanged player authorization still verifies because hash/epoch are absent;
# place_bet reads N directly from the live commitment.
submit_ed25519_and_place_bet(player_auth = auth, client_seed = client_seed)
assert round.server_seed_hash == sha256(seed_1)
assert round.epoch == E + 1
```

```
# Reveal the selected epoch and settle the forced loss.
rotate_server_seed(prior_seed = seed_1, new_hash = sha256(seed_2))
settle_outcome(claimed_payout = 0)
assert settled_outcome.payout == 0
release()
assert player_balance.balance + player_balance.locked_in_play ==
       player_claim_before - wager
assert game_exposure.total_collected == pool_ggr_before + wager
```

Every on-chain commitment and reveal check succeeds because the program proves consistency with the commitment installed after authorization, not that the player authorized that commitment.

Impact

The core commit-reveal path has a binary outcome: a win pays reserved, while a loss pays exactly zero in `programs/game-router-core/src/settlement/digest.rs:365-372`. `place_bet` has already transferred exactly wager from the player vault into bet escrow in `programs/game-router-ref/src/lib.rs:334-353`. On the forced-loss path, `release_reserved` transfers the full staked `bet_amount` from escrow into `pool_vault`, credits exactly that stake as GGR, and extinguishes the player's claim in `programs/house-pool/src/instructions/release_reserved.rs:162-223` and `programs/house-pool/src/instructions/release_reserved.rs:252-284`. Therefore, each manipulated round causes a deterministic player loss equal to 100% of wager. Repeating the attack across relayed bets can produce material aggregate player losses while every commitment, reveal, and settlement check succeeds on-chain.

Recommendation

The preferred fix is to bind player consent to the exact live commitment. Add `server_seed_hash` and epoch to `compute_player_auth_digest`, require clients to read and verify that on-chain commitment before signing, and have `place_bet` recompute the digest from the same live account values. Any rotation between authorization and execution must then invalidate the authorization. Keep the relayer and seed-rotation authority operationally separated as defense in depth.

As a second safeguard, increment `next_nonce` whenever `rotate_server_seed` installs a new commitment. This does not prevent disclosure of `client_seed`; it prevents a captured pending transaction from surviving the rotation. The player authorization inside that transaction binds the pre-rotation nonce N, while `place_bet` recomputes the digest using the live nonce. Advancing the live value to N + 1 therefore makes the captured authorization fail after rotation while leaving already-open rounds settleable from their snapshotted state. Clients with pending submissions must re-sign after rotation. Never reset or decrement the nonce, because reuse could make an older authorization valid again under a later commitment. This nonce invalidation is defense in depth and should not replace explicit commitment binding in the signed digest.

Status

Resolved

[H-04] game-router-core::verify_commit_reveal - Commit-reveal settlement allows outcome-dependent reveal aborts

Description

game-router-core presents CommitReveal as a provably-fair settlement primitive, but it guarantees only the integrity of a seed that the seed holder chooses to reveal. The verifier does not guarantee that the reveal occurs, even though the documented threat model claims that pre-commitment prevents the operator from selecting a profitable outcome in `programs/game-router-core/src/settlement/commit_reveal.rs:1-25`.

After bets are accepted, the game administrator knows both the committed server-seed preimage and every public `client_seed`, so it can calculate all outcomes before the players can. Revelation applies to an entire epoch: the administrator can reveal house-favorable epochs and let player-favorable epochs time out so that only the wagers are returned. It can make this selection increasingly granular by operating short or single-round epochs. This contradicts the reference integration's explicit invariant that the operator cannot cherry-pick winning rounds in `programs/game-router-ref/src/lib.rs:300-304`.

Vulnerability Details

The canonical core flow binds the plaintext `client_seed` into the player's authorization in `programs/game-router-core/src/settlement/player_auth.rs:35-72`. Core then derives the payout from that public value and the private server seed in `programs/game-router-core/src/settlement/digest.rs:262-285`. Meanwhile, ServerSeedCommitment records a single administrator that controls the server-seed commitment and contains no independent reveal or anti-abort state in `programs/game-router-core/src/state/server_seed_commitment.rs:36-60`.

The decisive limitation is in the verifier logic itself: `verify_commit_reveal()` begins after a server seed has already been supplied, checks its hash, derives the payout, and validates the claim in `programs/game-router-core/src/settlement/commit_reveal.rs:67-97`. There is no state transition for an absent reveal. Core nevertheless re-exports this raw verifier and the

associated commitment types as the reusable integration surface in `programs/game-router-core/src/lib.rs:94-103`, requiring every embedding game to invent the security-critical reveal and timeout lifecycle independently.

The reference adapter shows the direct consequence of that incomplete primitive. Only the commitment administrator can create the epoch reveal through `rotate_server_seed` in `programs/game-router-ref/src/lib.rs:175-205`. Settlement loads the resulting RevealedEpochSeed and invokes the core verifier in `programs/game-router-ref/src/lib.rs:513-565`; without that account, the verifier is unreachable. Instead, once the reveal deadline passes, `refund_stall` treats the absent reveal as refund eligibility, returns the wager 1:1, and permanently marks the round Refunded in `programs/game-router-ref/src/lib.rs:771-831`. A later reveal cannot restore the winning payout because settlement accepts only an Open round in `programs/game-router-ref/src/lib.rs:466-480`.

The attack uses regular flow. It does not require a captured authorization, relayer control, malformed accounts, transaction-ordering control, or a rare protocol state. The administrator can operate a one-round epoch, calculate its outcome, reveal a player loss, or wait through the reveal deadline and refund a player win before rotating to the next epoch. With multiple concurrent rounds, one reveal settles the whole epoch, so the administrator instead chooses according to the epoch's aggregate outcome; this changes selection granularity but does not remove the isolated, sequential, or short-epoch paths.

Impact is High because systematic suppression of winning payouts is repeatable across epochs, positions, and users and can cause material aggregate financial loss. Likelihood is High because the game administrator normally possesses the seed, observes every accepted round input, and controls epoch revelation; these are regular capabilities rather than additional prerequisites. No trusted-control-plane cap applies: the documented security model expressly promises that the operator cannot steer or censor outcomes, so selecting revelation according to known outcomes is outside the discretion entrusted to this role. High impact and High likelihood produce Critical severity.

Binding player authorization to the exact server commitment, as required by H-01, does not fix this issue. That protection prevents the administrator from replacing the authorized seed, but the administrator can still selectively withhold the already-bound seed after the bet is accepted. The two issues therefore have independent root causes and remediations.

Core already reserves `SettlementSource::Vrf`, but explicitly documents it as unwired in `programs/game-router-core/src/state/settlement_source.rs:18-35`. The reference integration therefore rejects every VRF settlement with VrfNotImplemented in `programs/game-router-ref/src/lib.rs:590-593`, leaving the administrator-controlled seed as the only on-chain re-derived randomness source.

Proof of Concept

Starting with a one-round epoch whose server seed S is known to the game administrator:

```
commit_server_seed(sha256(S))


# A normal player authorizes and opens a round with public client seed C.
place_bet(client_seed = C)
outcome = derive_commit_reveal_outcome(S, C, round, nonce, wager, reserved, edge)


if outcome == 0:
    # Preserve the player loss.
    rotate_server_seed(prior_server_seed = S, new_server_seed_hash = next_hash)
    settle_outcome(claimed_payout = 0)
else:
    # Suppress the player win.
    do_not_reveal_until(reveal_deadline)
    refund_stall()
    assert round.status == Refunded
```

```
    # A later valid reveal cannot reinstate the winning payout.
    rotate_server_seed(prior_server_seed = S, new_server_seed_hash = next_hash)
    assert settle_outcome(claimed_payout = reserved) == AlreadySettled
```

On player-losing epochs the administrator retains the wager through normal settlement. On player-winning epochs it returns the principal but avoids the payout, turning the nominally symmetric random process into an outcome-dependent settlement policy. A larger epoch applies the same decision to the aggregate of all rounds bound to that seed.

Impact

Players can have valid winning payouts systematically suppressed while house-favorable epochs continue to settle. Although an aborted round returns its wager, repeating the strategy across short epochs, users, and positions distorts the advertised payout distribution and can deprive players of a material aggregate amount of winnings.

Recommendation

Do not use administrator-controlled CommitReveal for production settlement against the shared LP bankroll. Fully implement the reserved `SettlementSource::Vrf` path in game-router-core and make it the canonical production randomness source.

Bind the player-authorized `client_seed`, game and round identifiers, player, wager, reserved payout, house edge, and nonce before requesting randomness. Persist the unique VRF request identity in canonical round state, accept only the first valid proof and output bound to that request, and derive the payout on-chain from the verified VRF output plus the already-bound client seed. The administrator must not be able to choose the request result, substitute another output, retry an unfavorable request, or gate access to a fulfilled result.

Core should provide the VRF request state, provider/account validation, proof or callback verification, outcome derivation, replay protection, and permissionless settlement transition so embedders do not implement these security-critical checks independently. If a request expires without a valid result, refund the wager, release its

reservation, and pause the game before accepting new bets. Because the client seed is fixed before the unpredictable VRF output exists, the player does not need a second reveal transaction.

Disable or migrate existing CommitReveal configurations before production use. Keeping CommitReveal available as an administrator-controlled alternative would preserve the same selective-abort and administrator-player collusion surface that VRF is intended to remove.

Status

Resolved

[H-05] game-router-core::ServerSeedCommitment - A game-wide nonce enables permissionless admission denial of service

Description

Every wager in a game consumes the same ServerSeedCommitment.next_nonce, which is a single monotonically increasing counter shared by all players in `programs/game-router-core/src/state/server_seed_commitment.rs:23-50`. Two players that authorize wagers against the same current nonce therefore race for one usable value: the first successful transaction advances the counter, while every other authorization for that nonce becomes invalid.

This produces a Medium-severity self-denial-of-service condition during ordinary concurrent use and also permits a funded player to turn the race into an active High-severity denial of service. The attacker can bypass the game UI and backend, continuously read the on-chain counter, sign direct micro-wagers, and submit them with competitive priority fees to invalidate pending honest authorizations.

Vulnerability Details

`place_bet` reads the live nonce and increments it before verifying the player's authorization in `programs/game-router-ref/src/lib.rs:284-324`. The signature digest binds the wager to this nonce in `programs/game-router-core/src/settlement/digest.rs:375-448`, but the nonce belongs to the game rather than to the player or authorization. Consequently, all players of the same game serialize through one shared authorization sequence.

If several players sign nonce N, only the first successfully executed wager can use it. Subsequent transactions recompute their authorization digest with the now-current nonce N + 1, fail signature verification, and roll back. The users must fetch the new value, sign again, and enter another race. Under ordinary bursts of concurrent betting this causes poor throughput, failed transactions, and repeated signing even without a malicious participant.

An attacker does not need privileged access or cooperation from an off-chain

component. A funded player can choose a unique round identifier, sign a minimum-size wager for the current nonce, and submit the Ed25519 verification and `place_bet` instructions directly. The house-pool accepts a one-base-unit wager as its minimum in `programs/house-pool/src/instructions/reserve_payout.rs:225-237`. One successful attacker transaction can invalidate any number of honest authorizations prepared for the same nonce, so the disruption can be amortized across a burst of victims.

The logical nonce race is isolated by `game_id`, but parallel attack streams are straightforward to automate. Moreover, each accepted bet writes the per-game commitment and passes several shared writable house-pool accounts, including `pool_config`, `game_exposure`, and `balance_snapshots`, in `programs/game-router-ref/src/lib.rs:997-1057`. Release and refund also write shared pool and game exposure accounts in `programs/game-router-ref/src/lib.rs:1179-1205` and `programs/game-router-ref/src/lib.rs:1268-1291`. Sustained high-priority admission traffic can therefore contend with settlement, release, and refund transactions for write access, potentially delaying resolution of wagers that were already accepted. It does not invalidate refunds logically, and a fixed blind submission cadence does not guarantee a complete stall; the attacker must keep synchronizing with on-chain state and win transaction ordering against honest retries.

The attack has a non-zero but realistic sustained cost. Every successful wager initializes a 204-byte RefRound at the attacker's expense in `programs/game-router-ref/src/lib.rs:986-995`. Its data size is 196 bytes plus the discriminator in `programs/game-router-ref/src/state.rs:97-118`, and neither the release nor refund context closes this account in `programs/game-router-ref/src/lib.rs:1158-1177` and `programs/game-router-ref/src/lib.rs:1231-1248`. The current rent-exempt minimum is therefore locked after each successful nonce consumption.

Using a rent-exempt minimum of 0.00231072 SOL for 204 bytes, two 5,000-lamport signature fees per transaction under the [current Solana fee schedule](https://solana.com/docs/core/fees/fee-structure), and a spot price of [$77.64 per SOL on July 22, 2026](https://www.coinbase.com/converter/sol/usd), the

lower-bound attacker cost is approximately:

- $0.18 per successful nonce consumption;
- at one successful transaction per minute, approximately $10.81 per hour, $259 per day, or $1,816 per week;
- $10,897 for one successful transaction every ten seconds for one week; or
- $108,973 for one successful transaction per second for one week.

These figures exclude priority fees and fees paid for transactions that lose the race, both of which increase the cost of a reliable stall. A one-minute cadence may still be effective against a slow game with predictable submission waves, while a fast game requires a denser stream. The estimate shows that a targeted week-long attack is economically plausible for a motivated player or competing game operator, even though complete suppression is not guaranteed at any fixed frequency.

At one successful transaction per minute, the most likely effect against a frequently used or fast game is recurring admission grief rather than a complete stall. Each attacker success invalidates every honest authorization that was prepared for the consumed nonce, causing up to one burst of failed wagers and re-signing per minute, but honest wagers can use subsequent nonces between attacker transactions. This cadence is also too sparse by itself to create material scheduler contention for settlement or refund accounts because it does not hold account locks between transactions. Against a slow game that collects or submits wagers only once every minute or several minutes, however, a synchronized attacker that observes and front-runs each submission wave may invalidate most pending authorizations and make the game practically unusable. A blind once-per-minute cadence cannot guarantee that result; achieving it requires timing against the honest submission pattern and paying sufficient priority fees.

The active variant has High impact because an attacker can repeatedly keep a selected game unavailable and can also delay terminal processing through shared-account contention faster than an operator can recover through ordinary retries or off-chain filtering. Its likelihood is Medium: the attack is permissionless, simple to automate, and

requires no UI or backend access, but sustained suppression requires funded transactions, state synchronization, priority fees, and success in transaction ordering. High impact and Medium likelihood produce High severity.

The naturally occurring variant is separately Medium severity. Its impact is Medium because concurrent players suffer failed submissions and severely degraded throughput but at least one transaction from each nonce race can succeed and an off-chain sequencer can provide a practical, although centralized, recovery path. Its likelihood is Medium because the race is deterministic once players concurrently authorize the same nonce, while meaningful service degradation requires sufficient same-game concurrency.

Proof of Concept

1. Initialize one game and fund two independent player accounts.
2. Read the game's current ServerSeedCommitment.next_nonce as N.
3. Have both players sign different wager authorizations that each bind nonce N.
4. Submit both Ed25519 verification plus `place_bet` transactions.
5. Observe that the first transaction succeeds and advances the shared nonce to N + 1, while the second transaction fails authorization because the handler recomputes its digest using N + 1.
6. Repeat the procedure with an attacker-controlled funded player and unique round identifiers, always signing the latest nonce and prioritizing the attack transaction ahead of pending honest wagers.
7. Observe that every attacker success invalidates all honest authorizations for the consumed nonce and that sustained submissions also compete with release and refund transactions for shared writable pool accounts.

Impact

Ordinary concurrent betting can cause a high failure rate, repeated wallet signing, and severely degraded throughput for all players of a game. A funded attacker can sustain the same condition deliberately, making a targeted game practically unavailable without interacting with its UI or backend. High-priority spam can additionally delay settlement, release, and refunds of accepted wagers through contention on shared pool

accounts, prolonging the period during which player wagers and protocol exposure remain unresolved.

Recommendation

Remove the game-wide sequential nonce from player authorization. Prefer a player-selected random `authorization_id` with sufficient entropy and derive the round PDA from (`game_id`, player, `authorization_id`). Bind the game id, player, authorization id, complete wager terms, server-seed commitment hash and epoch, and an expiry slot into the signed digest. Initializing the unique PDA then provides atomic replay protection without forcing unrelated players to share a counter.

As a simpler alternative, maintain a nonce PDA per (`game_id`, player). This restores cross-player parallelism but still serializes concurrent wagers from the same player, so a unique authorization id is preferable.

Make ServerSeedCommitment read-only during normal wager admission and write it only during seed rotation. Terminal paths should depend only on the round's snapshotted commitment and epoch data and should avoid unnecessary access to admission-specific accounts. In particular, remove the unused commitment account from `refund_stall` and, if the round snapshot and revealed epoch seed are sufficient, from settle_outcome.

Finally, reduce shared house-pool contention by avoiding writes to a global balance-snapshot ring on every wager and by sharding or pre-allocating per-game exposure capacity. Admission traffic should not share avoidable writable accounts with settlement, release, or refund. On-chain minimum-wager, open-bet, and congestion controls can raise the cost of residual spam, but off-chain firewall or UI rate limiting is not a sufficient mitigation for direct transactions.

Status

Resolved

[H-06] revenue-distributor::distribute_epoch - unrefreshable PoolTvlOracle reverts the crank for every profitable epoch, locking all revenue

Description

`distribute_epoch` sizes the insurance-bucket cap from a program-owned `PoolTvlOracle` account and rejects the crank if that oracle is older than roughly 24 hours. No instruction in the program creates or updates the oracle, and under Solana's ownership model no external program can write to a `revenue-distributor`-owned account. Because epochs are weekly, the oracle is already stale before the first profitable distribution, so `distribute_epoch` reverts with `StalePoolTvlOracle` for every epoch that has net profit - permanently trapping all accumulated USDC in `usdc_vault` in `programs/revenue-distributor/src/instructions/distribute_epoch.rs:330`.

Vulnerability Details

The oracle is loaded as a typed Anchor account (owner must be this program) and gated on a fail-closed staleness check, and it is read only on the profitable path:

```
// distribute_epoch.rs:121-125 - typed load REQUIRES owner == revenue_distributor + discriminator
pub pool_tvl_oracle: Box<Account<'info, PoolTvlOracle>>,

// distribute_epoch.rs:285-330 - the oracle is read ONLY when there is profit to split
if net_profit_usdc > 0 {
    // ...
    let pool_tvl = read_pool_tvl(&ctx.accounts.pool_tvl_oracle, clock.slot)?;
}

// distribute_epoch.rs:603-613 - fail-closed staleness gate (216_000 slots ≈ 24h)
let age_slots = current_slot.saturating_sub(oracle.last_update_slot);
require!(age_slots <= MAX_POOL_TVL_ORACLE_STALENESS_SLOTS,
RevenueError::StalePoolTvlOracle);
Ok(oracle.tvl)
```

No on-chain writer exists. A full-repository search for `PoolTvlOracle` / `tvl` / `last_update_slot` writers returns none: `initialize` stores only the oracle *key* as an `UncheckedAccount` in

`programs/revenue-distributor/src/instructions/initialize.rs:127-131,282`, and no program - including `house-pool` - creates the account or writes its fields. The only code in the repository that populates the oracle is the test harness, which plants it out-of-band with a capability that has no production equivalent:

```
// tests/test_revenue_distributor_insurance_pin_attest.ts:399-412
const oracleData = coder.encode("poolTvlOracle", {
  tvl: new BN(tvl),
  lastUpdateSlot: new BN(curSlot + 1_000_000), // "far-future => always fresh"
  updateAuthority: deploy.publicKey, bump: 0, reserved: Array(32).fill(0),
});
ctx.setAccount(oracle, { data: oracleData, owner: programId, ... }); // bankrun-only
primitive
// tests/test_revenue_distributor_claim_crank_cost_guards_e2e.ts:120-126 plants it
the same way
```

`setAccount` is a bankrun/litesvm test primitive that writes arbitrary bytes and an arbitrary owner into any account; it does not exist on mainnet or devnet. The tests deliberately set `last_update_slot` to a far-future slot so the staleness gate always passes, masking the absence of a real updater. On production Solana the account can only be populated through a program instruction, and the typed `Account<PoolTvlOracle>` load forces `owner == revenue-distributor`, so only `revenue-distributor` may write it - yet it exposes no such instruction. The tests derive the oracle as the program PDA `find([b"pool_tvl_oracle"], revenue_distributor)`, and a program PDA can only be created by its own program, so the account can never reach a valid, discriminator-correct state on-chain and every profitable `distribute_epoch` reverts. Even under the weaker reading where the oracle is an external keypair account seeded once via create-then-`assign`, it becomes immutable the moment ownership transfers to `revenue-distributor` and goes stale within 24 hours, while `DEFAULT_EPOCH_DURATION` is 7 days - so the first profitable distribution still reverts. The oracle read sits inside `if net_profit_usdc > 0`, so only zero-profit epochs, which distribute nothing, can be cranked - the exact inverse of the intended behaviour.

Proof of Concept

1. In the test suite, note the oracle is planted via bankrun `setAccount` with a far-future `last_update_slot`

(`tests/test_revenue_distributor_insurance_pin_attest.ts:399-412`); this capability does not exist on production Solana.

2. On a real deployment, `initialize` and accrue revenue through `receive_revenue` / `record_swept_revenue` so `accumulated_usdc > fixed_costs`, then wait for the epoch to expire (≥ 7 days for the default duration).

3. Call `distribute_epoch`. Because `net_profit_usdc > 0` it enters `read_pool_tvl`; the oracle PDA was never created or written by any instruction, so the typed load fails (or, if seeded once out-of-band, `current_slot - last_update_slot` exceeds `216_000` and it reverts `StalePoolTvlOracle`).

4. Observe that no `EpochRecord` is created, so no bucket entitlements exist and `claim_bucket` has nothing to pay.

5. Confirm no instruction can create or refresh the oracle; the revert is permanent for every profitable epoch and the accumulated USDC stays locked in `usdc_vault`.

Impact

The core distribution path is denied for every epoch with net profit, which is the normal operating case. No `EpochRecord` is produced, no bucket becomes claimable, and all accumulated revenue - not merely the fixed-cost residual - is locked in `usdc_vault` with no on-chain exit short of a program upgrade. Only zero-profit epochs, which distribute nothing, can be settled.

Recommendation

Add an owner-writable `update_pool_tvl` instruction to `revenue-distributor`, gated on `PoolTvlOracle.update_authority`, that creates the oracle PDA if absent and sets `tvl` and `last_update_slot = Clock::get()?.slot`, and invoke it within the staleness window before each crank - ideally in the same transaction as `distribute_epoch`, or via CPI from `house-pool` at settlement. Align `MAX_POOL_TVL_ORACLE_STALENESS_SLOTS` with the configured epoch cadence so an oracle written at settlement is not already stale by the next crank. Until such an instruction exists, `distribute_epoch` cannot settle any profitable epoch on-chain, and the test-suite coverage that plants a fresh oracle via `setAccount` does not reflect production behaviour.

Status

Resolved

[H-07] staking-veswoobz::* / buyback-burn::execute_twap_slice - SWOOBZ transfers omit the Token-2022 transfer-hook accounts and revert against the production hook-enabled mint - protocol-wide, breaking staking, unstaking, SWOOBZ reward claims, and the buyback/burn flywheel

Description

Across the protocol, every SWOOBZ transfer is built with a plain `transfer_checked` CPI that passes only `{source, mint, destination, authority}`. The production SWOOBZ mint carries an active Token-2022 `TransferHook` extension (set in `swoobz-token::initialize_mint`), and Token-2022 requires the hook's `ExtraAccountMetaList` plus the four extra accounts it declares to be present on every transfer of that mint. Because none of these programs supplies those accounts, Token-2022 cannot invoke the hook and the transfer reverts. This makes staking's locking (`create_position`), unlocking (`close_position`), and SWOOBZ reward claims/funding (`claim_gpr`, `claim_pow_fee`, `fund_gpr_reserve`) non-functional (`programs/staking-veswoobz/src/instructions/close_position.rs:110-122`), and it also breaks `buyback-burn::execute_twap_slice` (`programs/buyback-burn/src/instructions/execute_twap_slice.rs:197-209`), where the cranker deposits acquired SWOOBZ into the burn vault - so the buyback/burn flywheel cannot settle either. `revenue-distributor::transfer_swoobz_from_vault` has the same defect but is currently dead code. Only USDC-denominated paths (which have no transfer hook) are unaffected.

Vulnerability Details

The transfer CPI carries no hook accounts and never forwards `remaining_accounts`:

```
// close_position.rs:110-122 (create_position / claim_gpr / claim_pow_fee /
fund_gpr_reserve are identical)
let cpi_accounts = TransferChecked {
    from: staking_vault, mint: swoobz_mint, to: user_swoobz, authority:
staking_vault,
};
transfer_checked(
    CpiContext::new_with_signer(token_program, cpi_accounts, &signer_seeds), // no
```

```
.with_remaining_accounts(...)
    amount, decimals,
)?;
// accounts struct has NO transfer-hook program, NO ExtraAccountMetaList, NO
pause/HolderStatus PDAs
```

The production SWOOBZ mint is created with the hook extension, and the hook's `ExtraAccountMetaList` declares four required extra accounts:

```
// swoobz-token/src/instructions/initialize_mint.rs - mint has
ExtensionType::TransferHook (pinned to the hook program)
// transfer-hook/src/instructions/initialize_extra_account_metas.rs -
build_extra_metas() (EXTRA_META_COUNT):
//   0. pause_state PDA            seeds = [b"pause_state_v1"]
//   1. source HolderStatusV1 PDA  seeds = [b"holder_status_v1", source_owner]
//   2. dest   HolderStatusV1 PDA  seeds = [b"holder_status_v1", dest_owner]
//   3. instructions sysvar
```

On a hook-enabled mint, Token-2022 (spl-token-2022 v11) resolves the `ExtraAccountMetaList` PDA (`[b"extra-account-metas", mint]`) and the four declared extras, then CPIs into `hook::execute` - all of which must be present in the transfer instruction's account list, and none of which the staking transfers provide. Modern Token-2022 does not skip the hook; when the extras cannot be resolved the transfer reverts (the same failure the Orca V2 wrapper raises as `NoExtraAccountsForTransferHook`). `anchor_spl::token_interface::transfer_checked` is a thin wrapper that does not resolve or append hook accounts; the hook-aware path is `spl_token_2022::onchain::invoke_transfer_checked`. A repository-wide search confirms `staking-veswoobz` never references the transfer hook, `ExtraAccountMetaList`, `remaining_accounts`, or `with_remaining_accounts`. The LP-yield rail (`claim_lp_yield`) is unaffected because it moves USDC, which has no transfer hook.

The bug is masked by the test suite: `createToken2022MintBR` (`tests/bankrun_helpers.ts:444`) builds a plain base Token-2022 mint (only `createAccount` + `createInitializeMint2Instruction`, no extensions), and the staking lifecycle e2e (`tests/test_staking_veswoobz_lifecycle_pow_e2e.ts:254`) exercises `create_position` / `close_position` against that hook-less mint, so the plain

`transfer_checked` succeeds in tests while it reverts in production.

The identical defect exists in `buyback-burn::execute_twap_slice`, where the cranker deposits the SWOOBZ it acquired off-chain into the burn vault:

```
// buyback-burn/src/instructions/execute_twap_slice.rs:197-209
anchor_spl::token_interface::transfer_checked(
    CpiContext::new(                    // no .with_remaining_accounts(...)
        token_2022_program.key(),
        TransferChecked { from: cranker_swoobz, mint: swoobz_mint, to: swoobz_vault,
authority: cranker },
    ),
    cranker_swoobz_balance, SWOOBZ_DECIMALS,
)?;
// accounts struct: token_2022_program present, but NO hook program /
ExtraAccountMetaList / HolderStatus PDAs
```

A repository-wide search confirms none of the SWOOBZ-transferring programs (`staking-veswoobz`, `buyback-burn`, `revenue-distributor`) references the transfer hook, `ExtraAccountMetaList`, `remaining_accounts`, or `with_remaining_accounts`. The complete affected surface:

1. staking-veswoobz - `create_position` (user → staking vault): staking is blocked; no one can lock SWOOBZ.
2. staking-veswoobz - `close_position` (vault → user): unstaking is blocked; locked stake cannot be withdrawn.
3. staking-veswoobz - `claim_gpr` / `claim_pow_fee` (reserve/fee vault → claimant): SWOOBZ rewards cannot be claimed.
4. staking-veswoobz - `fund_gpr_reserve` (admin → reserve vault): the reward reserve cannot be funded.
5. buyback-burn - `execute_twap_slice` (cranker → burn vault): the buyback/burn flywheel cannot settle.
6. revenue-distributor - `transfer_swoobz_from_vault` (vault → recipient): dead code today; would fail if wired.

`anchor_spl::token_interface::transfer_checked` is a thin wrapper that does not

resolve or append hook accounts; the hook-aware path is `spl_token_2022::onchain::invoke_transfer_checked`. Every USDC path is unaffected (USDC has no transfer hook), which is why the type/CPI-module pairing is otherwise correct - the programs use the right module for SWOOBZ (`token_interface`), they simply never pass the hook's required accounts, so the failure is a revert (fail-closed), not a silent extension bypass.

Proof of Concept

1. Create the SWOOBZ mint via `swoobz-token::initialize_mint`, which activates the `TransferHook` extension, and initialize the hook's `ExtraAccountMetaList` (declaring pause + source/dest HolderStatus + instructions sysvar).
2. Attempt `create_position` to lock SWOOBZ. The `user_swoobz -> staking_vault transfer_checked` provides no hook accounts; Token-2022 cannot resolve the `ExtraAccountMetaList` and the transfer reverts - no one can stake.
3. Independently, if stake is present in the vault, call `close_position` after lock expiry. The `staking_vault -> user_swoobz transfer_checked` (lines 110-122) again omits the hook accounts and reverts - the stake cannot be withdrawn.
4. Observe the identical failure in `claim_gpr`, `claim_pow_fee`, and `fund_gpr_reserve` (all plain SWOOBZ `transfer_checked` with no hook accounts).
5. Run `buyback-burn::execute_twap_slice`: the USDC leg (`token::transfer`) succeeds, but Step 2's `token_interface::transfer_checked` moving the acquired SWOOBZ into `swoobz_vault` reverts for the same reason - the buyback slice can never settle.
6. Note the suites pass only because the test SWOOBZ mint (`createToken2022MintBR`) is built hook-less, so production semantics are never exercised.

Impact

The SWOOBZ side of the protocol is non-functional against the production mint. In `staking-veswoobz`, users cannot lock or unlock stake and no GPR/PoW reward can be claimed or funded; if stake ever reached the vault (for example through a temporarily hook-less mint), `close_position` would revert permanently and that stake would be locked with no recovery path. In `buyback-burn`, `execute_twap_slice` reverts on its SWOOBZ deposit leg; because the instruction is atomic, the USDC leg rolls back with it, so no slice can ever execute and the buyback/burn flywheel is entirely stalled (no

SWOOBZ can be acquired or burned). This is a core-functionality break stemming from the protocol's Token-2022 integration being validated against a simplified test mint rather than the extension-bearing production mint. Only USDC-denominated paths continue to function.

Recommendation

Route every SWOOBZ transfer through the hook-aware path so the CPI carries the required accounts. Either replace the `anchor_spl transfer_checked` calls with `spl_token_2022::onchain::invoke_transfer_checked(...)`, which resolves the `ExtraAccountMetaList` and appends the extra accounts automatically, or add the transfer-hook program, the `ExtraAccountMetaList` PDA, the pause-state PDA, and the source/destination `HolderStatusV1` PDAs to each instruction's account context and forward them via `CpiContext::new(...).with_remaining_accounts(...)`. Apply the fix to all SWOOBZ-transferring instructions: `staking-veswoobz::{create_position, close_position, claim_gpr, claim_pow_fee, fund_gpr_reserve}`, `buyback-burn::execute_twap_slice`, and `revenue-distributor::transfer_swoobz_from_vault` (before it is wired). Critically, rebuild the test SWOOBZ mint WITH the `TransferHook` extension (matching `initialize_mint`) across all three programs' suites so they exercise production semantics and this class of bug becomes visible. See Solodit #65237 (Pashov / Saffron) for the same hardcoded-missing-hook-accounts pattern.

Status

Resolved

Medium

[M-01] house-pool::deposit - Weak first-depositor share-inflation defense due to under-sized DEAD_SHARES

Description

The retail deposit handler mints `DEAD_SHARES = 1_000` LP-token units to a permanently locked PDA on the first deposit as a Uniswap-V2 `MINIMUM_LIQUIDITY` analog. However, the LP mint uses 6-decimal precision (matching USDC), so 1,000 units equals only ~$0.001 of locked value. This is 3-6 orders of magnitude weaker than the 18-decimal token assumption on which the original `MINIMUM_LIQUIDITY = 1000` pattern relies, leaving the pool exposed to the classical first-depositor share-price inflation attack.

Vulnerability Details

The bootstrap branch of `deposit::handler` computes the first depositor's shares as follows:

```
// programs/house-pool/src/instructions/deposit.rs (bootstrap branch)
let (user_shares, dead_shares_to_mint) = if total_supply_before == 0 {
    require!(amount > DEAD_SHARES, HousePoolError::DepositTooSmall);
    let user = amount
        .checked_sub(DEAD_SHARES)
        .ok_or(HousePoolError::ArithmeticOverflow)?;
    (user, DEAD_SHARES)
} else { ... };
```

With `DEAD_SHARES = 1_000` (declared in `state/constants.rs:59`) and `LP_TOKEN_DECIMALS = 6`, the seed lock equals only $0.001, which is negligible relative to any realistic first-deposit amount. A well-capitalized attacker races to be the first depositor with `amount = min_deposit = 1_000_000` (1 USDC), receiving `999_000` LP shares while the dead PDA holds only `1_000`. The attacker then transfers USDC directly into the pool vault ATA (bypassing `deposit::handler`) to inflate NAV without minting additional shares. Subsequent share mints use `shares = amount * T / NAV` (floor), so any depositor whose `amount * T < NAV` receives zero shares (reverts as `SharesRoundedToZero`) or loses dust in the round-down. The attacker's captured share of the manipulated NAV is `(A -`

`DEAD_SHARES) / T_final`, meaning the only cost is `DEAD_SHARES / A` (0.1% of the seed amount) - a negligible tax compared to the dust extracted from many subsequent depositors.

Proof of Concept

Numerical example with `A = 1e6` (attacker seed, 1 USDC), `D = 1e12` (direct donation, 1M USDC), and a whale victim `V = 2e12` (2M USDC):

- Attacker deposit: mints `999_000` shares for `A`, dead PDA locks `1_000` shares. `T = 1_000_000`. Vault = `1e6`.
- Attacker direct donation: Vault = `1e6 + 1e12 ≈ 1e12`. `T` unchanged.
- Whale deposit: `shares_whale = 2e12 * 1e6 / (1e6 + 1e12)` ≈ `1_999_998` (floors).
- Attacker withdraws `999_000` shares: recovers `999_000 * (1e6 + 1e12 + 2e12) / (1e6 + 1_999_998) ≈ (1 - 1_000/1_000_000) * (A + D) ≈ 0.999 * (A + D)`.

Attacker net position ≈ `-0.001 * D` = -$1,000 on a $1M attack - a tax orders of magnitude smaller than the dust captured across a realistic depositor stream. Any smaller-than-whale depositor whose `amount * T < NAV` reverts entirely with `SharesRoundedToZero`, giving the attacker a permanent DoS on small depositors.

Impact

Small and mid-size depositors are either denied service (share mint reverts to zero) or lose share-price rounding dust to the attacker. For large depositor volume, aggregated dust can exceed the attacker's $0.001-level DEAD_SHARES tax, making the attack net-profitable.

Recommendation

Raise `DEAD_SHARES` so that the seed lock has meaningful economic weight at 6-decimal precision. Two options that both eliminate the attack:

```
// Option A: raise DEAD_SHARES to 1e6 (one whole LP unit ≈ $1 seed lock).
pub const DEAD_SHARES: u64 = 1_000_000;

// Option B: keep DEAD_SHARES small, but enforce a bootstrap-only minimum deposit.
```

```
// e.g. require amount ≥ 100 USDC on the first deposit so the seeded NAV
// itself makes donation-based inflation uneconomical.
```

Option A is preferable - a single-point constant change with no interface impact.

Status

Resolved

[M-02] house-pool::deposit / house-pool::reserve_payout - Kelly bet capacity DoS via pending_unvested_at stamp-once flooding

Description

`deposit::handler` re-stamps `pool_config.pending_unvested_at = clock.unix_timestamp` on every deposit and accumulates each deposit's `amount` into `pending_unvested_amount`. `reserve_payout::handler` derives the Kelly betting cap from `effective_balance = pool_balance - pending_unvested_amount`. A griefer with modest capital can lock the pool's capacity indefinitely by depositing at intervals shorter than `MIN_DEPOSIT_HOLD_SECONDS = 600s`, keeping the entire pending bucket permanently unvested.

Vulnerability Details

The stamp-on-every-deposit logic in `deposit.rs:371-383`:

```
// programs/house-pool/src/instructions/deposit.rs:371-383
if pool_config.pending_unvested_at == 0
    || clock.unix_timestamp.saturating_sub(pool_config.pending_unvested_at)
        >= MIN_DEPOSIT_HOLD_SECONDS
{
    pool_config.pending_unvested_amount = 0;
}
pool_config.pending_unvested_amount = pool_config
    .pending_unvested_amount
    .checked_add(amount)
    .ok_or(HousePoolError::ArithmeticOverflow)?;
pool_config.pending_unvested_at = clock.unix_timestamp;
```

The reset condition compares against the timestamp of the most recent deposit, not the timestamp at which the current bucket was first opened. Any new deposit that lands within the 10-minute window resets the vesting clock for the whole accumulated bucket.

`reserve_payout.rs:184-197` then uses this bucket to shrink Kelly:

```
// programs/house-pool/src/instructions/reserve_payout.rs:184-197
```

```
let unvested = if pool_config.pending_unvested_at == 0
    || clock.unix_timestamp.saturating_sub(pool_config.pending_unvested_at)
        >= MIN_DEPOSIT_HOLD_SECONDS
{ 0 } else { pool_config.pending_unvested_amount };
let effective_balance = pool_balance.saturating_sub(unvested);
```

Proof of Concept

Attack sequence:

1. Attacker deposits `min_deposit = 1 USDC`. `pending_unvested_amount = 1 USDC`, timestamp = t.
2. Attacker waits 599s (< `MIN_DEPOSIT_HOLD_SECONDS`) and deposits another 1 USDC. Bucket does not reset; `pending_unvested_amount = 2 USDC`, timestamp = t + 599.
3. Attacker repeats every 599s. After N deposits, `pending_unvested_amount = N USDC` and the bucket has never vested.
4. `effective_balance` in `reserve_payout` is reduced by the full attacker deposit stream. Kelly cap shrinks proportionally, throttling house revenue.

The attacker's capital sits in the vault but is fully withdrawable at any time (withdrawal does not check the unvested bucket) - the griefing cost is only the transaction fees and the temporary opportunity cost of the capital.

The in-source comment in `deposit.rs:358-370` acknowledges this exact behavior:

```
// programs/house-pool/src/instructions/deposit.rs:358-370
// The known over-hold "capacity DoS" is the safe failure;
// the correct non-conservative fix is a bounded ring of
// {amount, deposited_at} tranches where each vests on its own
// clock (tracked follow-up), NEVER stamp-once.
```

Impact

House betting capacity can be suppressed indefinitely by a griefer whose only cost is transaction fees and the yield opportunity cost of the deposited USDC. The pool's

Kelly-sized `max_bet` collapses toward zero, degrading service for legitimate game bets and reducing house GGR.

Recommendation

Replace the single scalar `(pending_unvested_amount, pending_unvested_at)` with a bounded ring of `{amount, deposited_at}` tranches. Each tranche vests on its own clock, so a fresh deposit cannot reset the vesting timer of prior tranches:

```
pub struct PendingTranche {
    pub amount: u64,
    pub deposited_at: i64,
}
pub struct PoolConfig {
    // ...
    pub pending_tranches: [PendingTranche; N],
    pub pending_tranches_head: u8,
}
```

In `reserve_payout`, sum only tranches whose `deposited_at` is within `MIN_DEPOSIT_HOLD_SECONDS` of `clock.unix_timestamp`.

Status

Resolved

[M-03] house-pool::withdraw - Trailing-hour sliding-window under-count allows ~2× cap bypass at sub-window boundary

Description

The retail `withdraw::handler` enforces the ADR-004 hourly ($200K) and daily ($1M) velocity caps through a 2-sub-window sliding counter (`WithdrawalTracker`) whose trailing-usage estimate is a linear time-weighted average of the previous sub-window's total. The estimate implicitly assumes withdrawals in the previous sub-window were uniformly distributed across it. An attacker that concentrates withdrawals in the last second of a sub-window (before it rolls) can straddle the boundary and drain ~2× the stated hourly cap inside a real trailing hour. The in-source design comment in `withdraw.rs:301-307` explicitly claims the sliding counter "so the straddle is counted in full" - the numerical trace below shows only ~0.028% of the concentrated prior spend is actually counted.

Vulnerability Details

The trailing-usage estimator (`math/circuit_breaker.rs:199-211`):

```
// programs/house-pool/src/math/circuit_breaker.rs:199-211
pub fn trailing_carryover(prev_withdrawn: u64, position_in_window: i64,
window_seconds: i64) -> u64 {
    let pos = position_in_window.clamp(0, window_seconds);
    let remaining = (window_seconds - pos) as u128;
    let w = window_seconds as u128;
    let num = (prev_withdrawn as u128).saturating_mul(remaining);
    ((num + (w - 1)) / w) as u64   // ceil(prev * remaining / w)
}
```

Applied against the sub-window roll in `withdraw.rs:318-331`, the carryover assumes uniform temporal distribution of the previous sub-window's spend. When the actual spend is concentrated at the END of the previous sub-window (moments before the roll), the algorithm treats it as if it were spread across the full window, drastically under-counting the amount that is still inside the real trailing hour.

Proof of Concept

Constants: `ONE_HOUR = 3600`, `ADR004_HOURLY_LIMIT = 200_000_000_000` (= $200K micro-USDC), `ADR004_PER_TX_LIMIT = 50_000_000_000` (= $50K).

Starting state at T = 0: `{ current: 0, prev: 0, start: 0 }`.

Step 1 - attacker submits 4 × $50K withdrawals at T = 3599 (last second of the first sub-window). Each passes the hourly check (`current + carry + amount ≤ 200K`) because `prev = 0`. State after: `{ current: 200K, prev: 0, start: 0 }`.

Step 2 - attacker returns at T = 7199 (3600 s later, one second before the full-reset threshold):

- `hourly_elapsed = 7199`; `7199 < 2·ONE_HOUR = 7200` and `7199 ≥ ONE_HOUR = 3600` → single roll: `prev = 200K, current = 0, start = 3600`.
- `position = 7199 - 3600 = 3599`, `remaining = 1`.
- `carry = ceil(200_000_000_000 * 1 / 3600) = 55_555_556` micro-USDC ≈ $55.56.
- `prior = 0 + 55_555_556 ≈ $55.56`.
- Attacker can withdraw up to `200_000_000_000 - 55_555_556 = 199_944_444_444` micro-USDC ≈ $199,944.44 (via 4 more txs, last sized to $49,944.44).

Step 3 - measure true trailing-hour throughput at T = 7199:

- Trailing hour = `[3599, 7199]`.
- Contains $200,000 (at T = 3599) + $199,944.44 (at T = 7199) = ~$399,944.
- Design cap = $200,000. Ratio = 199.97%. Effective bypass ~2×.

The pattern is repeatable at every T = 3600·n + 3599, so peak trailing-hour throughput is sustained at ~2× the cap indefinitely. Identical math on the daily counter (24-hour window) yields peak trailing-24h throughput of ~$2M vs the stated $1M cap.

Impact

The load-bearing invariant `trailing_hour_throughput ≤ HOURLY_LIMIT` (documented in `withdraw.rs:299-308` as the reason for the sliding-window design) is violated by ~2×. Same violation on the daily cap. During adverse events (bet-loss cascade, pool solvency scare, bank-run), the fastest-exiting LP can extract at double the intended rate, forcing slower LPs to absorb proportionally more of subsequent NAV drops. No direct fund loss to other LPs (attacker still limited to their pari-passu share), but the throttle's designed blast radius is doubled.

Recommendation

Two viable fixes, in order of preference:

Option A - increase sub-window granularity. Replace the 2-sub-window counter with N sub-windows of `WINDOW / N` each (e.g. six 10-minute sub-windows per hour). The uniform-distribution error scales as `1/N`, so 6 sub-windows caps the bypass factor at ~1.17×.

```
pub struct WithdrawalTracker {
    pub hourly_buckets: [u64; N_HOURLY],   // e.g. [u64; 6]
    pub hourly_bucket_starts: [i64; N_HOURLY],
    // ... same for daily
}
```

Trailing usage = sum of any bucket whose start is within `WINDOW` seconds of `now`, plus a proportional slice of the oldest partial-overlap bucket.

Option B - exact accounting via a bounded ring of `{ amount, timestamp }` entries. Higher account rent, but the trailing-window total becomes exact and the sliding-window approximation is eliminated entirely.

Option C (interim mitigation) - halve the constants to compensate for the observed 2× worst-case bypass. Zero code change but tightens legitimate LPs unnecessarily.

Status

Resolved

[M-04] house-pool::* - Systemic PDA rent-lock: six account types init'd without an on-chain close path, with unbounded linear growth on two

Description

Six distinct PDA account types in `programs/house-pool/` are `init`'d during normal protocol operation but have no `close =` constraint and no dedicated close handler anywhere in the codebase. When each account's purpose is fully served - a game is deauthorized, an LP fully redeems, a player fully withdraws all balance, a tournament backstop is fully repaid, an epoch closes, or a compliance entry is permanently revoked - the on-chain state persists forever with its rent lamports permanently trapped. No authority tier (operational, treasury, governance, or guardian) can reclaim the rent; there is no code path that closes any of these accounts under any condition.

Two of the six (`EpochSnapshot`, `TournamentFundingRecord`) grow unboundedly and linearly with protocol usage - one new PDA per epoch and per tournament respectively, forever. The remaining four grow linearly with user count. Individual per-account rent is small (~$0.20-$1.50) but the pattern is systemic and cumulative: over years and at protocol scale, aggregate trapped rent reaches material dollar amounts, and permanent on-chain account count growth imposes ongoing storage cost on the Solana state.

This finding covers the rent-lock aspect of `deauthorize_game`; the separate same-program-key-ban consequence of that account remaining alive on-chain is documented under its own dedicated finding, since it produces a distinct functional issue beyond pure rent trapping.

Vulnerability Details

The same pattern recurs across seven `init`-declared account types in `programs/house-pool/`: each declares `payer = ...` and `seeds = [...]` but omits the `close = ...` constraint, and no sibling handler ever closes the account under any condition. The affected sites are `GameExposure` at `authorize_game.rs:70-77` (INIT_SPACE = 221 bytes, ~$0.50 rent per game - `deauthorize_game.rs:93` only mutates `exposure.authorized = false`), `EpochSnapshot` at `take_epoch_snapshot.rs:36-43` (59 bytes, ~$0.20 rent per snapshot, permissionless init, one PDA per epoch forever),

`TournamentFundingRecord` at `top_up_tournament.rs:96-107` (73 bytes, ~$0.22 rent per record, explicitly documented as never-closed at `repay_backstop.rs:12-13`), `LpPosition` at `open_lp_position.rs:37` and `open_institutional_lp_position.rs:62` (171 bytes, ~$0.44 rent per LP, never closed even when `settled_shares == 0 && claimable_yield == 0 && redemption_pending_shares == 0`), `PlayerBalance` + `PlayerVault` at `player_open_vault.rs:45-56` (98 + 165 bytes, ~$0.68 rent per player, never closed even after `balance == 0 && locked_in_play == 0`), `PlayerExposure` at `open_player_exposure.rs:20-27` (93 bytes, ~$0.25 rent per player, never closed after `active_reservations == 0 && current_exposure == 0`), and `InstitutionalAllowlist` at `open_institutional_allowlist.rs:71` (118 bytes, ~$0.31 rent per institutional LP, never closed on permanent revocation via `manage_institutional_allowlist.rs:85-94`).

`GameExposure` is representative of the shape at every site - `init` with a payer, seeded PDA layout, no `close =` constraint anywhere on the struct - and every one of the other six sites is a syntactic variation on this same template. The unbounded-growth axis is different from the trapped-rent axis: `EpochSnapshot` and `TournamentFundingRecord` accumulate one new PDA per epoch / per tournament for the protocol's entire lifetime (so their aggregate rent and on-chain state grow without ceiling), while the remaining four accumulate one per user, growing linearly with user count; the `EpochSnapshot` data is redundantly emitted in `EpochSnapshotTaken` events so on-chain persistence is not even load-bearing for observability:

```
// programs/house-pool/src/instructions/authorize_game.rs:70-77  (representative pattern)
#[account(
    init,
    payer = authority,
    space = 8 + GameExposure::INIT_SPACE,
    seeds = [GAME_EXPOSURE_SEED, game_program.key().as_ref()],
    bump,
    // ← no `close = ...` constraint here, and no sibling handler closes it either
)]
pub game_exposure: Box<Account<'info, GameExposure>>,
```

Impact

Roughly $51K of SOL rent is permanently trapped across a 3-year mid-sized deployment (dominated by exited-player accounts), with no authority able to reclaim any of it. `EpochSnapshot` and `TournamentFundingRecord` growth further inflates on-chain state without ceiling. No USDC principal is at risk.

Recommendation

Add explicit close paths for each account type, gated on the invariants that prove the purpose is fully served:

(a) `GameExposure` - either add `close = authority` on the `game_exposure` account in `deauthorize_game.rs` so rent returns to the operational authority and the same program key can be re-authorized cleanly, or add a dedicated `re_authorize_game` handler that flips `authorized = true` back on an existing deauth'd PDA under the same signer/state constraints as `authorize_game`.

(b) `EpochSnapshot` - add a governance-gated `close_epoch_snapshot(epoch: u64)` handler that closes any snapshot older than N epochs, sweeping rent to a designated treasury. Snapshot data is already event-emitted, so on-chain retention is redundant.

(c) `TournamentFundingRecord` - if off-chain `tournament_id` is guaranteed globally unique (e.g., monotonic counter or UUID), extend `repay_backstop` with `close = pool_authority` when the final repayment brings `funding_record.amount == 0`. If IDs may recycle, keep as-is and document expected cumulative operational-treasury rent cost.

(d) `LpPosition` - add a user-callable `close_lp_position` handler with `close = owner`, gated on:

```
require!(lp_position.settled_shares == 0, ...);
require!(lp_position.claimable_yield == 0, ...);
require!(lp_position.redemption_pending_shares == 0, ...);
require!(depositor_lp.amount == 0, ...);
```

Emit a `LpPositionClosed` event with lifetime stats so off-chain audit history is preserved.

(e) `PlayerBalance` + `PlayerVault` + `PlayerExposure` - add a user-callable `player_close_vault` handler that closes all three atomically with `close = player`, gated on:

```
require!(player_balance.balance == 0, ...);
require!(player_balance.locked_in_play == 0, ...);
require!(player_exposure.active_reservations == 0, ...);
require!(player_exposure.current_exposure == 0, ...);
require!(player_vault.amount == 0, ...);
```

The `PlayerVault` SPL token account is closed via a `token::close_account` CPI (mirroring the escrow-close pattern in `void_bet.rs:244`).

(f) `InstitutionalAllowlist` - add a compliance-authority-gated `close_institutional_allowlist(lp_owner)` handler with `close = compliance_authority`, gated on `allowlist_entry.allowed == false` AND (optionally) verification that the LP holds no institutional shares.

Adopting all six recommendations closes the rent-lock at each account type, reclaims aggregate protocol-lifetime SOL, and caps unbounded on-chain state growth.

Status

Resolved

[M-05] house-pool::activate_circuit_breaker + initialize_balance_snapshots - Balance snapshot ring warm-up window disables loss-based circuit-breaker checks for the first ~10-12 hours after ring initialization

Description

The `BalanceSnapshots` ring buffer is zero-initialized by `initialize_balance_snapshots.rs`, with all 24 snapshot slots holding `PoolSnapshot { balance: 0, timestamp: 0 }`. `get_balance_hours_ago` in `math/circuit_breaker.rs` skips uninitialized slots and returns 0 when no in-tolerance snapshot exists. Both loss checks in `circuit_breaker_evaluate` are gated on the returned baseline being `> 0`, so an empty or sparse ring silently disables both rapid-loss and catastrophic-loss protection. Additionally, `activate_circuit_breaker.rs:105` intentionally passes `initial_pool_balance: 0` to disable the emergency-shutdown branch in this handler (documented at lines 98-100 as delegated to the guardian `emergency_pause` path). The net effect is that during the ring's warm-up window - from ring init until the first snapshot ages ~12 hours - only the aggregate-exposure overflow check remains active, and any drain that does not inflate open exposure counters proceeds without breaker intervention.

Vulnerability Details

Both loss checks in `math/circuit_breaker.rs` gate on `> 0` - an empty ring returns 0 and silently skips the check:

```
// math/circuit_breaker.rs:95-110  (catastrophic-loss check)
if pool_balance_24h_ago > 0 {
    let loss_24h = pool_balance_24h_ago.saturating_sub(pool.pool_balance);
    let threshold_24h = (pool_balance_24h_ago as u128)
        .checked_mul(config.catastrophic_loss_threshold_bps as u128)
        .unwrap_or(0)
        / (BPS_SCALE as u128);
    if loss_24h >= threshold_24h as u64 {
        return (CircuitBreakerAction::PauseAll, "catastrophic_loss");
    }
}
```

```
// math/circuit_breaker.rs:126-140  (rapid-loss check)
if pool_balance_1h_ago > 0 {
    let loss_1h = pool_balance_1h_ago.saturating_sub(pool.pool_balance);
    let threshold_1h = (pool_balance_1h_ago as u128)
        .checked_mul(config.rapid_loss_threshold_bps as u128)
        .unwrap_or(0)
        / (BPS_SCALE as u128);
    if loss_1h >= threshold_1h as u64 {
        return (CircuitBreakerAction::ReduceMaxBets, "rapid_loss");
    }
}
```

Snapshots are written only by `reserve_payout.rs:390` and `take_epoch_snapshot.rs:150`, and the `record_snapshot` writer throttles to one write per `interval_seconds` (3600s). Under the tolerance of `2 × interval = 7200s`, the 1h lookup finds a valid snapshot only once slot 0 exists, and the 12h lookup finds one only after slot 0 is between 10 and 14 hours old. If the pool is opened for bets immediately after `finalize_pool` without cranking `take_epoch_snapshot` during the warm-up window, the ring stays empty until the first bet's `reserve_payout`, and the catastrophic-loss ceiling is silently disabled for ~10 hours after that first write. A compromised authorized game (whether via a mistakenly-authorized wallet-as-game key, or a legitimate game program later upgraded to malicious code) that paces winning-bet drain just under the 5% rapid-loss threshold can cumulatively extract up to ~40% of the pool over the warm-up window before the 12h check activates and trips.

Proof of Concept

Attack sequence, assuming pool at $10M vault, `rapid_loss_threshold_bps = 500` (5%), `catastrophic_loss_threshold_bps = 1500` (15%), and an authorized game whose game-authority PDA is under attacker control:

1. Pool completes multi-step init → LPs deposit $10M → operator authorizes the attacker's game (multisig-gated but achieved via `authorize_game` registration slippage or a legitimately-approved-then-compromised program).
2. `initialize_balance_snapshots` has been called; ring is empty. Operator does not crank `take_epoch_snapshot`.

3. T = T_0: attacker's game places a first bet. `reserve_payout.rs:390` writes `slot 0 = { balance: $10M, timestamp: T₀ }`. Rapid-loss check now has a baseline; catastrophic check still returns 0.
4. T = T_0 + Δ: attacker's game "wins" the bet. `release_payout` drains `pool_vault` by the payout amount. No ring write on release.
5. Attacker paces drain at 4.9% per hour (staying just under the 5% rapid-loss threshold). Each hour a new snapshot lands at the throttled ring interval, capturing the drained balance. Baseline drifts downward.
6. Anyone calls `activate_circuit_breaker` during hours 1-10. Rapid-loss check compares to the previous hour's snapshot: 4.9% < 5% → no trip. Catastrophic-loss check: `get_balance_hours_ago(now, 12)` returns 0 (no snapshot old enough) → check silently skipped.
7. T = T_0 + 10h: pool balance = \$10M × 0.951^{10} ≈ \$6.05M (~39.5% drained). Next `activate_circuit_breaker` call: 12h lookup finds slot 0 (aged 10h, within the 7200s tolerance of the 12h target). Returns \$10M baseline. Loss = 39.5% > 15% → `PauseAll` trip.

Attacker net: ~39.5% pool drain (~\$3.95M on a \$10M pool) vs. the intended 15% catastrophic ceiling. Additional ~\$2.45M of realized LP loss attributable to the warm-up gap.

Impact

Under a compromised-authorized-game threat model (a legitimate game later upgraded to malicious code, or a mistakenly-authorized wallet-as-game key), a fresh pool is drainable by up to ~40% during the ring's first ~10 hours vs. the intended 15% catastrophic cap - an additional ~25 percentage points of realized LP principal loss over a one-time window per pool lifetime. The gap is bounded (closes automatically once the ring has 12 hours of snapshots) and requires the compromised game to be an early-mover on the pool, so material-loss risk is concentrated at deployment.

Recommendation

Ship an operator SOP requiring `take_epoch_snapshot` to be cranked hourly for 12 hours after `initialize_balance_snapshots` and before authorizing any game, so the ring is

fully populated before bet flow starts. Optionally add a code fix: pre-populate the ring at `initialize_balance_snapshots` with the current vault balance across all 24 slots at synthetic timestamps `now - i × 3600`, or add a `high_water_mark` fallback baseline in `circuit_breaker_evaluate` when the ring returns 0. Combined with an `executable` constraint on `authorize_game`'s `game_program` account, the compromised-game precondition is materially harder to trigger.

Status

Resolved

[M-06] transfer-hook::set_holder_status + Token-2022::SetAuthority - Sanctions evasion by transferring token account ownership to a clean wallet before transfer

Description

The transfer-hook enforces sanctions by reading `FLAG_SANCTIONED` on the `HolderStatusV1` PDA derived from the source token account's OWNER (bytes 32..64) - not from the wallet that signed the transfer. `set_holder_status(FLAG_SANCTIONED)` sets this bit in the hook's own state but does NOT invoke Token-2022's `FreezeAccount` on the sanctioned wallet's SWOOBZ token accounts. Consequently, a sanctioned wallet can execute Token-2022's `SetAuthority(AccountOwner)` to re-point their token account's owner to a clean wallet before initiating a transfer, and the hook - which re-derives `HolderStatusV1` from the new (clean) owner on each transfer - will approve the movement.

Vulnerability Details

The hook's source-side sanctions read at `programs/transfer-hook/src/instructions/execute.rs:317-326, 373-384`:

```
// programs/transfer-hook/src/instructions/execute.rs:317-326, 373-384
let source_owner: Pubkey = {
    let src_token_data = source_token.try_borrow_data()?;
    Pubkey::new_from_array(owner_bytes)   // bytes 32..64 of the source token account
};
if source_flags & FLAG_SANCTIONED != 0 {
    return err!(HookError::SourceSanctioned);
}
```

The wallet-side sanctions mutation at `programs/transfer-hook/src/instructions/set_holder_status.rs:88-152` writes only to the hook's `HolderStatusV1` PDA and does NOT CPI to Token-2022 to freeze the wallet's SPL accounts:

```
// programs/transfer-hook/src/instructions/set_holder_status.rs:88-152
pub fn set_holder_status_handler(ctx: Context<SetHolderStatus>, new_flags: u8) ->
```

```
Result<()> {
    reject_durable_nonce(...)?;
    require_squads_authority(...)?;
    status.status_flags = updated_flags;   // writes to HolderStatusV1 only
    // No CPI to spl_token_2022::freeze_account.
}
```

Token-2022's `SetAuthority` instruction only rejects the mutation if the SPL token account itself is `AccountState::Frozen` (its native frozen field). It has NO awareness of the transfer-hook's `HolderStatusV1.status_flags`. A wallet that is sanctioned at the hook layer but not frozen at the Token-2022 layer can freely re-point their token account's owner.

Proof of Concept

Prerequisites: attacker wallet `W_bad` holds SWOOBZ in token account `T`. Multisig has just sanctioned `W_bad`. Attacker sees the pending or landed `set_holder_status` transaction.

1. Multisig lands `set_holder_status(wallet=W_bad, new_flags=FLAG_SANCTIONED)`. `HolderStatusV1[W_bad].status_flags |= 0b_0000_0001`.
2. Attacker frontruns (or races in the next block) with `spl_token_2022::instruction::set_authority(T, AccountOwner, Some(W_clean))`. `W_bad` signs; Token-2022 checks `T.state != Frozen` - true (multisig hasn't called `FreezeAccount`) - succeeds. `T.owner = W_clean`.
3. Attacker as `W_clean` (permissionless) calls `initialize_holder_status(wallet=W_clean)`. `HolderStatusV1[W_clean].status_flags = 0`.
4. Attacker as `W_clean` calls `spl_token_2022::transfer_checked(T → any_destination, full_balance)`. Hook fires; reads T.owner bytes → `W_clean`; derives `HolderStatusV1[W_clean]` → tier 0, no sanctions bit → transfer approved.

Full balance moved. Sanctions defeated in one additional Token-2022 instruction.

Impact

Regulatory / compliance impact: on-chain sanctions enforcement is silently ineffective unless the multisig couples every `set_holder_status(FLAG_SANCTIONED)` with a Token-2022 `FreezeAccount` on every SWOOBZ token account the sanctioned wallet holds. This coupling is not documented in the codebase and not enforced programmatically. If ops misses any of the sanctioned wallet's SWOOBZ token accounts, or if the sanctioning tx is not atomic with the freezing txs, the wallet escapes with full balance. Not a direct fund-loss vector against other holders - loss is regulatory (failed OFAC/AML enforcement) and reputational.

Recommendation

Two viable options:

Option A (in-handler coupling) - modify `set_holder_status` to accept the sanctioned wallet's SWOOBZ token accounts as `remaining_accounts` and CPI `spl_token_2022::freeze_account` on each of them when `FLAG_SANCTIONED` is being set. Requires the caller to enumerate the wallet's token accounts, but ensures atomic sanction + freeze:

```
if (new_flags & FLAG_SANCTIONED) != 0 && (old_flags & FLAG_SANCTIONED) == 0 {
    for token_account_info in ctx.remaining_accounts.iter() {
        // Verify token_account_info.mint == SWOOBZ_MINT
        // Verify token_account_info.owner == wallet.key()
        // CPI: spl_token_2022::freeze_account(...) as freeze authority
    }
}
```

Option B (operator SOP + monitor) - document that `set_holder_status(FLAG_SANCTIONED)` MUST be preceded by (or bundled with) Token-2022 `FreezeAccount` calls on every SWOOBZ token account the sanctioned wallet holds. Deploy an off-chain monitor that alerts when a wallet has `HolderStatusV1.FLAG_SANCTIONED` set but any of their SWOOBZ token accounts is not Token-2022-frozen.

Option A is stronger (on-chain enforcement) but requires the caller to enumerate token accounts. Option B is simpler but relies on ops discipline and monitoring.

Status

Resolved

[M-07] game-router-core::compute_player_auth_digest - Core player authorizations are replayable across game deployments

Description

The reusable settlement library does not give embedding games a deployment-scoped player-consent domain. `compute_player_auth_digest` binds only `game_id`, `round_or_bet_id`, player, wager, `max_payout`, `client_seed`, and nonce in `programs/game-router-core/src/settlement/digest.rs:375-413`. It omits the executing program id, router config PDA, house-pool game authority, settlement source, commitment context, and rules identity. An authorization signed for one game can therefore authorize the same wager in another authorized deployment that recreates the signed fields.

Vulnerability Details

`verify_player_auth` recomputes this fixed digest internally and exposes no argument through which an embedding game can add its program id, config PDA, game authority, commitment, or rules identity in `programs/game-router-core/src/settlement/player_auth.rs:45-72`. An integrator can impose an application-specific namespace through `game_id` or add a separate authorization scheme, but the canonical core verifier neither enforces such namespacing nor provides a deployment identity that another game cannot copy.

An attacker who obtains house-pool authorization for a duplicate or semantically equivalent game can select the same `game_id`, reproduce an unused `round_or_bet_id` and current nonce, and submit a captured authorization with the same player, wager, maximum payout, and client seed. The core CPI builders then allow that authorized game to collect the wager and reserve its payout without linking those money-path calls to the deployment for which the player signed in `programs/game-router-core/src/cpi/builders.rs:44-117`.

To turn the replay into player loss, the attacker can replay authorizations that resolve as losses under the clone's independently committed state or reviewed rules. Repeating this across captured authorizations or multiple authorized duplicates can consume a

material share of the player's house-pool balance. Impact is High, while likelihood is Low because every clone must receive house-pool authorization and pass the platform's operational review. High impact and Low likelihood produce Medium severity.

Impact

An authorized clone can use player signatures intended for another game to place unintended wagers against the player's house-pool balance. Replaying matching authorizations that lose under the clone can transfer a material share of the player's balance to the house pool without the player consenting to that deployment.

Recommendation

Move deployment binding into game-router-core itself. Extend `compute_player_auth_digest` and `verify_player_auth` to bind an immutable deployment domain, the executing program id, router config PDA, house-pool game authority, settlement source, commitment context, and reviewed rules/paytable identity.

Status

Resolved

[M-08] game-router-core::compute_player_auth_digest - Signed wagers cannot be cancelled or expire

Description

The player-authorization preimage contains the game, round, player, wager, maximum payout, client seed, and nonce, but no expiry slot in `programs/game-router-core/src/settlement/digest.rs:375-392`. A relayer can therefore retain a signed wager and submit it long after the player intended to abandon it, provided its nonce remains current.

Vulnerability Details

`verify_player_auth` accepts no expiry parameter and verifies only the timeless fields in `programs/game-router-core/src/settlement/player_auth.rs:45-72`. There is also no player-controlled instruction that cancels a pending authorization or advances a player-scoped nonce.

The global nonce provides only incidental invalidation: another successful bet advances it before the retained signature is submitted. In a quiet or relayer-serialized game, the same nonce may remain current indefinitely. With sufficient available balance and an unused round id, the relayer can submit the old signature and open the exact wager without further participation from the player. This issue does not require commitment rotation, seed replacement, or outcome manipulation: delayed execution under the unchanged commitment is sufficient.

The round does not preserve the authorization time. Instead, the program reads the clock at execution in `programs/game-router-ref/src/lib.rs:398` and assigns that value to `opened_at` in `programs/game-router-ref/src/lib.rs:411`. The reveal and refund deadlines are then calculated from that new execution time in `programs/game-router-ref/src/lib.rs:412-421`. Consequently, no existing deadline limits how long the off-chain signature remains usable.

Proof of Concept

1. Fund a player vault, initialize a game and commitment, and read the current global nonce N.
2. Have the player sign the canonical wager digest for N, but do not submit place_bet.
3. Advance the validator clock by an arbitrary duration without placing another bet in that game.
4. Submit the original Ed25519 verification instruction followed by `place_bet` using the retained signature.
5. Observe that the wager is accepted, the player balance is debited, and `opened_at` reflects step 4 rather than the time at which the authorization was signed.

Impact

A relayer can execute a wager after the player believes the request has been abandoned or cancelled. If the stale wager loses, the player loses the amount of that individual wager despite no longer having current intent to place it.

Recommendation

Add a player-selected `expiry_slot` to `compute_player_auth_digest` in `programs/game-router-core/src/settlement/digest.rs:405-448`. Extend `verify_player_auth` in `programs/game-router-core/src/settlement/player_auth.rs:45-72` to accept that signed value and reject the authorization when `Clock::get()?.slot > expiry_slot`. Enforcing expiry in the library verifier ensures every embedding game performs the check before player authorization succeeds and funds can be collected.

As defense in depth, make `rotate_server_seed` monotonically advance `next_nonce` in `programs/game-router-ref/src/lib.rs:219-230` so authorizations cannot survive a commitment rotation. This does not replace `expiry_slot`: a stale game that does not rotate would otherwise leave the signature valid indefinitely. A player-scoped nonce with an explicit cancellation instruction would additionally allow selective revocation without withdrawing the player's balance.

Status

Resolved

[M-09] game-router-ref::place_bet - Invalid max_payout bounds break settlement and admission controls

Description

The bet-opening path accepts any positive wager and `max_payout` in `programs/game-router-ref/src/lib.rs:275-282` without enforcing the economic range required by the selected settlement model and the game's admitted multiplier. A CommitReveal payout below the wager creates a round that can never settle, while a payout above the registered maximum multiplier creates liability beyond the limit approved for that game.

Vulnerability Details

The lower-bound mismatch occurs because `place_bet` collects the wager, reserves the caller-provided maximum payout, and persists the round before settlement validation in `programs/game-router-ref/src/lib.rs:334-423`. The CommitReveal outcome function later requires reserved >= wager in `programs/game-router-core/src/settlement/digest.rs:288-299`. A positive pair such as wager = 2 and `max_payout` = 1 therefore opens successfully but always fails settlement with InvalidAmount.

An accepted sub-1x round raises pool-wide, per-game, and per-player exposure and creates a Reservation in `programs/house-pool/src/instructions/reserve_payout.rs:251-368`. Because settlement cannot consume it, the exposure remains allocated until `refund_stall` becomes available. An unrevealed epoch is refundable after five minutes; a revealed epoch can remain open until the full one-hour deadline in `programs/game-router-ref/src/lib.rs:766-831`. Repeating the flow can temporarily consume betting capacity and the game authority's escrow and Reservation rent budget.

The upper-bound mismatch occurs at the same trust boundary. `authorize_game` records `max_multiplier_bps` as the maximum payout multiplier for the game in `programs/house-pool/src/instructions/authorize_game.rs:92-128`. Although

`reserve_payout` uses this value to calculate the largest allowable stake, it never checks whether the supplied `max_payout` is bounded by `bet_amount` * `max_multiplier_bps` / 10_000 in `programs/house-pool/src/instructions/reserve_payout.rs:225-307`. The codebase already defines that exact relationship in `calculate_reserved_payout` at `programs/house-pool/src/math/kelly.rs:82-95`, but the money path does not call it or enforce an equivalent inequality.

As long as the arbitrary payout fits the independent aggregate, game, player, and available-liquidity caps, the handler stores it as the Reservation's source of truth. A valid winning outcome may then pay up to that amount because `release_payout` checks only that the payout does not exceed the recorded Reservation in `programs/house-pool/src/instructions/release_payout.rs:212-239`. The registered multiplier consequently affects Kelly stake sizing but does not constrain the liability it is supposed to describe.

For the canonical CommitReveal model, an oversized payout does not alter expected value: the verifier proportionally reduces its win probability in `programs/game-router-core/src/settlement/digest.rs:310-369`. It does, however, bypass the reviewed variance and per-game payout limit. Reaching a significant payout requires both an approved integration to supply inconsistent economics and a corresponding winning outcome. The sub-1x branch requires funded repeated bets to cause meaningful disruption. Both branches therefore have Medium impact and Medium likelihood, producing Medium severity.

Proof of Concept

Using the existing game-router settlement harness, exercise both missing bounds. Values in the second case are micro-USDC (six decimals).

```
# Lower bound: accepted at open, impossible to settle.
place_bet(wager = 2, max_payout = 1, signed_by_player = true)
assert reservation.bet_amount == 2
assert reservation.max_payout == 1
reveal_epoch_seed()
assert settle_outcome(claimed_payout = 0) == InvalidAmount
```

```
assert ref_round.status == Open



# Upper bound: accepted above the admitted 2x multiplier.
pool_balance = 1_000_000_000_000       # $1,000,000
authorize_game(max_multiplier_bps = 20_000, house_edge_bps = 200)
place_bet(
    wager = 2_500_000_000,              # $2,500
    max_payout = 150_000_000_000,       # $150,000
    signed_by_player = true,
)



assert calculate_reserved_payout(2_500_000_000, 20_000) == 5_000_000_000
assert reservation.max_payout == 150_000_000_000
assert game_exposure.current_exposure increased by 150_000_000_000



settle_with_valid_winning_outcome(payout = 150_000_000_000)
release()
assert player_received == 150_000_000_000
```

With the default 15% per-player and 30% per-game exposure limits, the second reservation fits both caps even though it is 60x the wager and 30x the admitted payout ceiling.

Impact

Sub-1x rounds can temporarily occupy shared exposure and consume the game authority's rent budget while remaining impossible to settle. Legitimate bets may be rejected until the rounds become refundable and a caller processes them.

Oversized payouts bypass the maximum multiplier approved for a game. A valid win can impose a reserved and payable obligation substantially above the reviewed per-bet limit, increasing bankroll variance and exposing LP funds up to the independent exposure caps.

Recommendation

Validate the complete payout range before consuming the commitment nonce, collecting the wager, or reserving liquidity. For CommitReveal rounds, require `max_payout` >= wager. For every admitted game, enforce the upper bound with checked u128 arithmetic:

```
max_payout * 10_000 <= wager * game_exposure.max_multiplier_bps
```

Expose the settlement-specific checks through a canonical game-router-core helper and invoke it from every embedding game before money-path CPIs. Mirror the upper bound in house-pool::`reserve_payout`, where GameExposure and the actual Reservation inputs are both available, so an approved game cannot bypass its admission parameters. If `calculate_reserved_payout` is reused, replace its narrowing u128 as u64 conversion with a checked conversion that fails closed on overflow.

Status

Resolved

[M-10] house-pool::withdraw - Pool-wide WithdrawalTracker singleton enables trivial-cost cross-LP DoS on the retail exit path

Description

`WithdrawalTracker` is a singleton PDA (`seeds = [b"withdrawal_tracker"]`), shared by every retail LP. The ADR-004 hourly ($200K), daily ($1M), and per-tx ($50K) caps are enforced globally, not per-LP. An adversarial LP that holds ~$200K in retail LP shares can fill the pool-wide hourly cap in a single burst, blocking every other retail LP from withdrawing for ~1 hour. The griefing cost is only Solana transaction fees; the attacker recovers 100% of their own capital and can re-deposit and repeat, sustaining a rolling DoS on the retail exit path at trivial cost.

Vulnerability Details

The tracker's singleton nature (`state/withdrawal_tracker.rs:16-19`):

```
// programs/house-pool/src/state/withdrawal_tracker.rs:16-19
/// Singleton withdrawal velocity tracker.
///
/// PDA Seeds: [b"withdrawal_tracker"]
#[account]
pub struct WithdrawalTracker { ... }
```

And the shared read/write in `withdraw.rs:57-62`:

```
// programs/house-pool/src/instructions/withdraw.rs:57-62
#[account(
    mut,
    seeds = [WITHDRAWAL_TRACKER_SEED],
    bump = withdrawal_tracker.bump,
)]
pub withdrawal_tracker: Box<Account<'info, WithdrawalTracker>>,
```

There is no per-LP quota. The velocity check (`is_within_hourly_limit`, `is_within_daily_limit`) treats the shared tracker as one global budget. Institutional exits via `execute_institutional_redemption` bypass the tracker entirely (verified by grep - `is_within_*_limit` has exactly one caller: `withdraw.rs`), so only retail LPs suffer the DoS.

Proof of Concept

Attack sequence:

1. Attacker holds ~$200K in retail LP shares (either seeded directly or via prior deposits).
2. Attacker submits 4 × $50K withdrawals at T = 0. State: `{ current: 200K, prev: 0, start: 0 }`.
3. Every other retail LP that attempts a withdrawal in [T = 0, T ≈ 4500] is rejected with `WithdrawalExceedsHourlyLimit`. For any withdrawal of `w`, the pass condition is `carry + w ≤ 200K`, requiring `position ≥ 3600·(1 − (200K − w)/200K)`. For `w = $50K` (max per tx), the earliest allowed time is T = 3600 + 900 = 4500 s (~75 min after the attacker's burst). For any nonzero `w`, the block persists for at least a full hour.
4. Attacker re-deposits their $200K (no tracker interaction on deposit) and repeats the cycle every ~90 min, sustaining permanent DoS at 16 cycles/day.

Cost per cycle: 4 withdrawal txs + 1 deposit tx ≈ 5 × 5000 lamports ≈ $0.005. Attacker's capital is never lost, only cycled.

Impact

All retail LP withdrawals can be blocked for ~1 hour per cycle at negligible cost, sustainable 24/7 with a single ~$200K stake. In calm markets this is pure griefing, but under adverse events (bet-loss cascade, insolvency scare, run-on-the-bank) honest LPs are trapped and forced to absorb NAV drops they would otherwise have exited before - escalating the impact to material financial harm. Institutional LPs are unaffected because `execute_institutional_redemption` bypasses the tracker entirely.

Recommendation

Two viable directions:

Option A - per-LP tracker. Change `WithdrawalTracker` to a PDA seeded by `[LP_WITHDRAWAL_TRACKER_SEED, withdrawer.key().as_ref()]`. Each LP has their own hourly/daily budget. Rent overhead: one small PDA per LP. Eliminates cross-LP DoS entirely; the caps become per-user, matching how they are colloquially described.

Option B - LP-share-weighted quotas. Keep the singleton tracker but bound each withdrawal by `min(HOURLY_LIMIT, HOURLY_LIMIT_BPS · vault · lp_share_fraction)`. Pro-rates the pool-wide cap by the withdrawer's ownership so a small LP cannot exhaust another LP's fair share.

Option C - accept the DoS surface but document it explicitly. If the design intent is a truly pool-wide throttle (i.e. the pool as a whole cannot bleed faster than $200K/hr regardless of who is withdrawing), the current behavior is correct, and this finding should be re-classified as "informational - cross-LP DoS is a design tradeoff". In that case update the ADR-004 documentation to say so clearly.

Option A is the standard fix and preserves per-LP economics; option B is smaller code churn if per-LP PDA rent is undesirable.

Status

Resolved

[M-11] house-pool::authorize_game / house-pool::deauthorize_game - Missing executable constraint on user-controlled game_program seed allows wallet-as-game authorization → full pool drain

Description

`authorize_game::handler` accepts `game_program: UncheckedAccount<'info>` with no type discipline - no `executable` check, no owner check against a BPF loader, no address allowlist. The pubkey is used as a PDA seed for `game_exposure` and, more importantly, is written into `game_exposure.game_program`, which becomes the sole spend-authority credential for that game slot in `reserve_payout`, `void_bet`, and `burn_bet_fee`. If the operational multisig accidentally passes a wallet keypair as `game_program` (a normal-looking operator mistake - wallet addresses and program IDs are both 32-byte pubkeys), the wallet holder gains full game-authority spend rights and can drain the pool through the legitimate happy-path handlers up to `max_game_exposure_ratio_bps · vault` per slot. The in-source CHECK comment (`"No authority or funds are derived from it"`) is factually incorrect and actively obscures the drain path from future reviewers.

Vulnerability Details

`authorize_game.rs:63-77`:

```
// programs/house-pool/src/instructions/authorize_game.rs:63-77
/// CHECK: The game program being admitted to the registry. Used only as a
/// PDA seed for game_exposure (GAME_EXPOSURE_SEED + game_program.key())
/// and recorded in exposure.game_program. No authority or funds are
/// derived from it; identity is enforced indirectly by the PDA seed
/// constraint on game_exposure.
pub game_program: UncheckedAccount<'info>,

#[account(
    init,
    payer = authority,
    space = 8 + GameExposure::INIT_SPACE,
    seeds = [GAME_EXPOSURE_SEED, game_program.key().as_ref()],
    bump,
```

```
)]
pub game_exposure: Box<Account<'info, GameExposure>>,
```

The stored key becomes the authority credential in every downstream handler that spends against the game slot. From `reserve_payout.rs:53-61`:

```
// programs/house-pool/src/instructions/reserve_payout.rs:53-61
#[account(
    mut,
    seeds = [GAME_EXPOSURE_SEED, game_authority.key().as_ref()],
    bump = game_exposure.bump,
    constraint = game_exposure.authorized
        @ HousePoolError::GameNotAuthorized,
    constraint = game_exposure.game_program == game_authority.key()
        @ HousePoolError::GameProgramMismatch,
)]
pub game_exposure: Box<Account<'info, GameExposure>>,
```

Where `game_authority: Signer<'info>` - a Solana signer, which a wallet keypair holder can produce directly (no CPI required). The identical missing constraint exists in `deauthorize_game.rs`.

Note on seed-collision analysis: the PDA-seed byte prefix `GAME_EXPOSURE_SEED` guarantees a distinct namespace within the program, so cross-PDA-type collisions require SHA-256 collision (infeasible). The vulnerability is not the seed structure itself - it is the missing type constraint on the value being used as the seed.

Proof of Concept

Attack sequence (post-authorization of a wallet keypair as `game_program`):

1. Attacker generates a wallet keypair `W` (or convinces the multisig to authorize an attacker-controlled key via social engineering / operational error).
2. Multisig calls `authorize_game(game_program = W.pubkey(), params)`. Handler succeeds; `game_exposure.game_program = W.pubkey()`, `authorized = true`.
3. Attacker crafts a transaction where `game_authority = W`, signed by W's keypair directly. Calls `reserve_payout(bet_amount, max_payout = MAX_PAYOUT, player =`

`attacker_player_pda, bet_nonce = 0`):

- Passes `game_exposure.game_program == game_authority.key()` (both equal `W.pubkey()`).
- Reserves `max_payout` up to `max_game_exposure_ratio_bps · vault`.

4. Attacker as `W` calls `collect_bet(...)` settling the bet in the player's favor. Payout credited to `player_balance`.
5. Attacker as the player calls `player_withdraw`. USDC exits the vault.
6. Attacker repeats with fresh `bet_nonce` values until the per-game exposure cap is exhausted.

Total drain per authorization ≈ `max_game_exposure_ratio_bps · vault`. For a $10M pool at 20% per-game ratio, that is $2M per mistaken authorization.

The attack does not require compromising a legitimate game program, does not require an upgrade path, and does not require any post-authorization action from the multisig - it is a one-shot drain from a single misclick during onboarding.

Impact

Catastrophic (full per-game exposure drain) conditional on a single authorization mistake by the operational multisig - wallet addresses and program IDs are visually indistinguishable (both 32-byte base58 pubkeys), and the in-source CHECK comment actively misdirects operators by stating no authority is derived from the account. No existing gate catches the mistake, and even for a correctly-identified executable program the same drain becomes available after any post-authorization upgrade if the program's upgrade authority is not frozen.

Recommendation

Add an `executable` constraint on `game_program` in both `authorize_game.rs` and `deauthorize_game.rs`, and correct the CHECK doc-comment:

```
/// CHECK: The game program being admitted. MUST be an executable Solana
/// program - a wallet or data account would grant its keypair holder full
/// game_authority spend rights (drain path via reserve_payout → collect_bet).
/// The address itself is admitted at multisig discretion; the executable
```

```
/// constraint below enforces the minimum type discipline.
#[account(
    constraint = game_program.executable
        @ HousePoolError::GameProgramNotExecutable,
)]
pub game_program: UncheckedAccount<'info>,
```

Optionally stronger - also constrain the account owner to a known BPF loader (defense-in-depth against exotic account types that report `executable = true` for non-program reasons):

```
constraint = game_program.owner == &solana_program::bpf_loader_upgradeable::id()
    || game_program.owner == &solana_program::bpf_loader::id()
    @ HousePoolError::GameProgramInvalidOwner,
```

Additionally, add a new `HousePoolError::GameProgramNotExecutable` (and `GameProgramInvalidOwner` if adopting the strong form) variant, and update the mirrored comment in `deauthorize_game.rs` for consistency.

Design-level follow-up (not required for this finding to be resolved, but recommended for operational SOP): require that game programs have `upgrade_authority == None` at authorization time, or codify the multisig review of the game program's upgrade authority as part of the onboarding checklist. This closes the post-authorization-upgrade variant of the same drain path.

Status

Resolved

[M-12] staking-veswoobz::initialize_vaults - Raw system_instruction::create_account (not top-up + allocate + assign) allows permissionless bootstrap DoS via 1-lamport pre-fund of the vault PDAs

Description

`create_and_init_vault` uses raw `system_instruction::create_account`, which reverts with `AccountAlreadyInUse` if the target account has any lamports > 0 or is not System-owned. The three vault PDAs (`staking_vault`, `gpr_reserve_vault`, `pow_fee_vault`) are derived deterministically from the deployed program ID + constant seeds - anyone who observes the program deploy can compute the addresses immediately. An unpermissioned attacker can send 1 lamport to any one of the three PDAs before `initialize_vaults` is executed by the deploy authority; the whole instruction then reverts atomically, and staking-veswoobz becomes unbootable. Recovery requires a code patch (switch to `top-up + allocate + assign`) plus a program-upgrade multisig ceremony, meaning hours-to-days of downtime and a forced launch-time incident. The sibling `swoobz-token/initialize_mint.rs:342-349` explicitly documents this exact attack class and applies the correct fix - the mitigation pattern is already known to the team, but was not reused here.

Vulnerability Details

`programs/staking-veswoobz/src/instructions/initialize_vaults.rs:175-186` - the vulnerable call:

```
// programs/staking-veswoobz/src/instructions/initialize_vaults.rs:175-186
// 1. Create account owned by Token-2022
invoke_signed(
    &anchor_lang::solana_program::system_instruction::create_account(
        payer.key,
        vault.key,
        lamports,
        space,
        &spl_token_2022::ID,
    ),
    &[payer.clone(), vault.clone(), system_program.clone()],
    &[signer_seeds],
```

```
)?;
```

`programs/swoobz-token/src/instructions/initialize_mint.rs:342-349` - the team's own documentation of the attack class, applied in the sibling program:

```
// programs/swoobz-token/src/instructions/initialize_mint.rs:342-349
/// PRE-FUND-TOLERANT (top-up -> allocate -> assign, NOT raw create_account): the
mint
/// address is a public compile-time constant, so anyone can send it lamports with NO
/// signature; a raw create_account reverts AccountAlreadyInUse on any lamports > 0
and
/// would let a keyless attacker grief the one-shot genesis. `allocate`/`assign`
tolerate
/// a pre-funded account and STILL require the mint to SIGN - which only the deployer
can
/// (an attacker holds no mint key) - so an attacker can never pre-allocate/assign
the
/// mint to inject an extension. That signer requirement keeps genesis atomic against
/// injection; the top-up path only removes the lamport-squat DoS.
```

`programs/swoobz-token/src/instructions/initialize_mint.rs:389-427` - the correct `transfer → allocate → assign` implementation already in use in the same codebase.

The attack sequence flows as follows. Once the team publishes the staking-veswoobz program binary (a public on-chain event visible to any observer), an attacker or an MEV bot monitoring new-program-deploy transactions derives the three vault PDAs via `find_program_address` - a purely off-chain computation requiring no signer, no permission, and no cost beyond a single RPC call. The attacker then sends 1 lamport (approximately $0.0000002 in SOL) to any one of the three PDAs; a `system_instruction::transfer` requires only the payer's signature, and the recipient (a PDA) does not need to sign to receive lamports. Once the team's multisig subsequently executes `initialize_vaults` (typical delay between program deploy and init for a Squads ceremony is hours to days), the `create_account` CPI reverts with `AccountAlreadyInUse` (SystemError code 0). Because all three vault creations happen inside the same instruction, the failure is atomic and every vault fails together, leaving every downstream fund-moving instruction dead (`create_position`, `close_position`, `claim`, `harvest`, `claim_epoch_reward` all require the vaults to exist).

Impact

Staking-veswoobz becomes unbootable at genesis; recovery requires a code patch, a multisig-approved program upgrade, and a fresh `initialize_vaults` attempt - realistic downtime hours to days. Every dependent program that reads the staking-veswoobz vaults (`revenue-distributor`, `buyback-burn`) sees account-not-found, and launch-time LP incentive campaigns gated on active staking are blocked. The finding escalates to HIGH if the deployment plan revokes upgrade authority at genesis (permanent bricking of the program instance) or if downstream programs hard-code the staking-veswoobz program ID (fresh deploy cascades into a full ecosystem re-bootstrap).

Recommendation

Replace `create_and_init_vault` with the same `transfer → allocate → assign` pattern used at `swoobz-token/initialize_mint.rs:389-427`:

1. Compute the correct account length dynamically using `ExtensionType::try_calculate_account_len::<Token2022Account>(&[TransferFeeAmount, TransferHookAccount])` so the account is sized correctly for the SWOOBZ Token-2022 mint's account-side extensions.
2. Compute `rent_lamports = Rent::get()?.minimum_balance(account_size as usize)`.
3. Do a `system_instruction::transfer` from `payer` to `vault` for `rent_lamports.saturating_sub(vault.lamports())` (skipped when zero - tolerates any pre-squatted balance).
4. Do `system_instruction::allocate(&vault.key, account_size)` via `invoke_signed` with the vault's PDA seeds (the vault PDA signs, which only the program can produce - attackers cannot pre-allocate).
5. Do `system_instruction::assign(&vault.key, &spl_token_2022::ID)` via `invoke_signed` with the same seeds.
6. Call `spl_token_2022::instruction::initialize_account3` as today.

Add an integration test that pre-funds one of the vault PDAs with 1 lamport before calling `initialize_vaults`, and asserts the bootstrap still succeeds - proving the DoS window is closed.

Status

Resolved

[M-13] revenue-distributor::report_fixed_costs / distribute_epoch - reported fixed-cost USDC has no withdrawal path and accrues permanently in the vault

Description

The revenue distributor subtracts reported fixed costs from gross revenue to compute the distributable net profit, but the physical USDC corresponding to those costs is moved into `usdc_vault` in full and is never allocated to any bucket nor withdrawable by any instruction. Over successive epochs, an unbounded amount of USDC accretes in the vault with no on-chain exit in `programs/revenue-distributor/src/instructions/distribute_epoch.rs:182-190`.

Vulnerability Details

The full gross revenue enters `usdc_vault`, but only `net_profit` is ever made claimable; the `fixed_costs` slice is left behind with no exit path:

```
// receive_revenue.rs:138-203 - full gross enters the vault
usdc_vault += G;  accumulated_usdc += G;

// report_fixed_costs.rs:54-82 - no token movement, only a counter (bounded <=
accumulated_usdc)
fixed_costs_current_epoch += C;

// distribute_epoch.rs:182-190, 509-593 - only net_profit becomes claimable; vault
balance untouched
net_profit = accumulated_usdc - fixed_costs;   // = G - C
Σ usdc_distributions == net_profit;            // the C slice is never bucketed
accumulated_usdc = 0;

// claim_bucket.rs:188-201 - the ONLY USDC-out path, pays exactly the recorded split
transfer_usdc_from_vault(usdc_vault, dest, amount = usdc_distributions[idx]);
```

`transfer_usdc_from_vault` (`programs/revenue-distributor/src/helpers.rs:62-80`) is the sole USDC-out helper and is called only by `claim_bucket`; no reserve-sweep, account-close, or admin-withdraw instruction exists across the program's sixteen handlers. The `fixed_costs` residual is therefore inert - it cannot be claimed (bucket allocations are capped at `net_profit`) and is recoverable only through a program

upgrade. Likelihood is High because reporting costs is the intended, documented flow, while impact is bounded to the protocol's own retained revenue with no user under-payment, so it is scoped Medium.

Proof of Concept

1. Register a `CostReport` source and complete one epoch in which sources push a gross amount `G` into `usdc_vault` via `receive_revenue`, so `accumulated_usdc == G` and the vault balance increases by `G`.
2. Before distribution, call `report_fixed_costs(C)` with `0 < C <= G`, setting `fixed_costs_current_epoch = C`. No tokens move.
3. Call `distribute_epoch`. Observe `net_profit == G - C`, `Σ usdc_distributions == G - C`, `accumulated_usdc` reset to `0`, and the vault balance unchanged at `+G`.
4. Claim every bucket for the epoch. Total USDC leaving the vault equals `G - C`.
5. Observe the vault retains `C` USDC. Enumerate the program's instructions and confirm none can transfer that `C` out; it remains for every subsequent epoch, growing by each epoch's reported costs.

Impact

The USDC set aside for fixed costs is neither distributed nor withdrawable, so the value the protocol reserves to cover operating costs cannot be accessed on-chain to actually pay those costs. The trapped balance grows without bound in proportion to `lifetime_fixed_costs`, permanently reducing the protocol's usable capital unless a program upgrade introduces a withdrawal path.

Recommendation

Confirm the intended treatment of the fixed-cost portion. If it is meant to fund operations, add a governance-gated withdrawal that transfers from `usdc_vault` to an operations treasury, bounded by `lifetime_fixed_costs` minus the amount already withdrawn, so it can never encroach on recorded bucket entitlements (which the solvency invariant guarantees remain fully backed). If the retention is intentional, document it explicitly and confirm the program retains an upgrade path so the balance is not permanently inaccessible.

Status

Resolved

[M-14] revenue-distributor::withdraw_transfer_fees - always reverts because the SWOOBZ mint's withheld authority is the swoobz-token PDA, not the distributor config PDA

Description

`revenue-distributor::withdraw_transfer_fees` harvests the SWOOBZ Token-2022 withheld transfer fees by signing the `withdraw_withheld_tokens_from_mint` CPI as the `distributor_config` PDA. Token-2022 accepts that CPI only if the signer equals the mint's configured `withdraw_withheld_authority`, but that authority was set at genesis to a PDA of the *swoobz-token* program, a different address that the distributor can never sign for. The instruction therefore reverts on every call, so the distributor's advertised transfer-fee revenue stream is non-functional in `programs/revenue-distributor/src/instructions/withdraw_transfer_fees.rs:82`.

Vulnerability Details

The signer and the mint's required authority are PDAs of two different programs and can never coincide:

```
// swoobz-token/src/instructions/initialize_mint.rs:460-464 - sets the mint's
withheld authority
initialize_transfer_fee_config(
  Some(&fee_config_authority.key()),   // transfer_fee_config_authority
  Some(&withdraw_authority.key()),     // withdraw_withheld_authority
);
// withdraw_authority seeds = [b"withdraw_authority", mint]  → PDA of swoobz_token
(AE84…)

// revenue-distributor/src/instructions/withdraw_transfer_fees.rs:71-82 - signs the
withdraw CPI
authority: distributor_config.to_account_info(),
signer_seeds = [b"distributor_config", bump];  // PDA of revenue_distributor (3Nnt…)
withdraw_withheld_tokens_from_mint(cpi_ctx)?;  // Token-2022 rejects: signer != mint
authority
```

`b"withdraw_authority" + mint` under program `AE84kxXaZ9oAfbT57KKrDEZ9tZ9kP3CzNQ88EzgFaAuM` and `b"distributor_config"` under

program `3NntTMzbUtz4n36uBy2EiseZd94npvrFBb8eMpNUZtbD` are necessarily distinct addresses - different seeds and different program ids. swoobz-token exposes no `set_authority` to ever repoint the withheld authority (its only instructions are `initialize_mint`, `propose/execute/cancel_fee_update`, `reduce_anti_bot_fee`, `withdraw_fees`), so the mismatch is permanent. The only working harvest path is `swoobz-token::withdraw_fees`, which signs with the correct `[b"withdraw_authority", mint]` PDA and sends fees to `mint_config.treasury` (`programs/swoobz-token/src/instructions/withdraw_fees.rs:86-102,55`) - the token treasury, not the distributor's `swoobz_vault`. Test coverage confirms this: `tests/test_swoobz_token_withdraw_fees_attest.ts` exercises the swoobz-token path and passes, while `withdraw_transfer_fees` has no test anywhere in the repository because it cannot execute. Compounding the confusion, `receive_revenue` rejects the `TransferFee` source type with `UseWithdrawTransferFeesInstead` (`programs/revenue-distributor/src/instructions/receive_revenue.rs:108-111`), directing operators to the always-reverting instruction.

Proof of Concept

1. Deploy `swoobz-token` and `revenue-distributor` at their configured program ids and run `initialize_mint`, which sets the SWOOBZ mint's `withdraw_withheld_authority` to `find_program_address([b"withdraw_authority", mint], swoobz_token)`.
2. Accrue some withheld transfer fees on the mint (any taxed SWOOBZ transfer, then `harvest_withheld_tokens_to_mint`).
3. Call `revenue-distributor::withdraw_transfer_fees`. The CPI is signed by `find_program_address([b"distributor_config"], revenue_distributor)`, which is not the mint's withheld authority, so Token-2022 rejects it and the instruction reverts.
4. Confirm the fees are still collectible only through `swoobz-token::withdraw_fees`, which delivers them to `mint_config.treasury`, never to the distributor's `swoobz_vault` or `accumulated_lucky`.

Impact

The distributor can never harvest SWOOBZ transfer fees, and `accumulated_lucky` is never credited from this path. No funds are lost - the fees remain fully collectible by `swoobz-token::withdraw_fees` into the token treasury - but a documented revenue

stream is broken and operators are actively routed to an instruction that always fails. The impact is bounded today because the distributor's SWOOBZ settlement rail is itself unwired (`swoobz_distributions` is gated to `[0;6]` in `distribute_epoch`, `transfer_swoobz_from_vault` is never called, and `swoobz_vault` has no drain), but it becomes a live revenue-loss-of-accounting issue the moment that rail is activated.

Recommendation

Decide which program owns the withheld authority and make the code consistent with it. Either set the SWOOBZ mint's `withdraw_withheld_authority` to the distributor's `distributor_config` PDA at genesis so `withdraw_transfer_fees` can sign, or remove `revenue-distributor::withdraw_transfer_fees` entirely and have the distributor obtain harvested fees through `swoobz-token::withdraw_fees` with the treasury pointed at (or swept into) the distributor's `swoobz_vault`. Two programs must not both claim the same mint's withheld authority - only one PDA can hold it. Also update the `receive_revenue` `UseWithdrawTransferFeesInstead` guidance to reference the path that actually works.

Status

Resolved

[M-15] house-pool::assign_lp_quota_basis- Unauthenticated third-party write to another LP's withdrawal-quota counters allows targeted exit denial

Description

assign_lp_quota_basis moves withdrawal-rate entitlement between retail LP positions and carries the sender's already-spent window usage with it. The sender is authenticated by a Signer; the receiver is not authenticated. Any wallet can therefore charge its own spent withdrawal quota against an arbitrary retail LP, reducing or removing that LP's ability to withdraw. The instruction was added by the M-10 remediation (audit/REMEDIATION-2026-08.md) and does not exist at the audited baseline.

Vulnerability Details

The to_position account has no signer, no has_one and no opt-in flag. Its only binding is a derivation off its own stored owner field:

```
#[account(
    mut,
    seeds = [LP_POSITION_SEED, to_position.owner.as_ref(), &[TRANCHE_RETAIL]],
    bump = to_position.bump,
    constraint = to_position.tranche == TRANCHE_RETAIL
        @ HousePoolError::InvalidLpPosition,
    constraint = to_position.key() != from_position.key()
        @ HousePoolError::InvalidLpQuotaBasisAssignment,
)]
pub to_position: Box<Account<'info, LpPosition>>,
```

Anchor deserializes all accounts before evaluating constraints, so the self-referential seed expression executes normally. With an explicit bump, the check reduces to key == create_program_address([LP_POSITION_SEED, stored_owner, [TRANCHE_RETAIL]], stored_bump), which every position written by open_lp_position satisfies by construction. The constraint establishes that the account is a genuine LpPosition of this program. It does not establish which wallet the position belongs to, and the discriminator does not help either, since every LpPosition shares one.

The handler then writes spent usage onto that account:

```
{
    let to = &mut ctx.accounts.to_position;
    to.record_withdrawal_quota_hourly(moved_hourly)?;
    to.record_withdrawal_quota_daily(moved_daily)?;
    to.credit_quota_basis(amount)?;
}
```

record_withdrawal_quota_hourly and record_withdrawal_quota_daily are bare checked_adds with no clamp against the receiver's own quota. withdraw enforces those same counters on the only retail exit path:

```
require!(
    is_within_lp_window_quota(usdc_out, hourly_used, hourly_quota),
    HousePoolError::WithdrawalExceedsLpHourlyQuota
);
```

where is_within_lp_window_quota(amount, used, quota) is used + amount <= quota. Injecting used >= quota blocks the victim entirely, because withdraw separately requires usdc_out > 0.

The compensating credit_quota_basis(amount) does not offset the harm. The only consumer of quota_basis_shares is quota_weight:

```
pub fn quota_weight(&self, live_balance: u64) -> u64 {
    live_balance.min(self.quota_basis_shares)
}
```

For a straight depositor basis == live, since basis is credited at the minted amount in deposit and debited at the burned amount in withdraw. That makes min(live, basis + amount) == live, so the credit buys no additional quota while the carried usage lands in full.

The carried usage is also independent of the size of the assignment. carry_used(used, amount, basis_before) is ceil(used * amount / basis_before), so amount == basis_before moves 100% of the sender's usage however small basis_before is. withdraw takes

shares_to_burn: u64, so an attacker can reduce their own basis to a single share unit and still transfer their whole accumulated spend.

Small LPs are not the only exposure. quota_weight is live.min(basis), so any position with a zero basis sits at LP_WITHDRAWAL_QUOTA_FLOOR regardless of size. lp_position.rs describes that population directly: "an ordinary secondary-market sale moves shares without touching basis, the buyer's later burn saturates against a basis that is already zero, and no handler of ours runs on a raw SPL transfer to correct it."

A multi-million-dollar secondary-market buyer therefore holds a $1 daily quota until someone assigns them basis, and assign_lp_quota_basis is the documented repair path for that case.

Proof of Concept

Setup is a $500,000,000 pool and a victim sitting at the quota floor, which covers any zero-basis position regardless of size. The attacker needs one permissionless open_lp_position, min_deposit of USDC, and rent that is recoverable afterwards. reject_durable_nonce inspects only instruction 0, so the cycle fits in a single transaction.

The cycle is three instructions. Deposit min_deposit + 1, which credits basis at the minted amount. Withdraw LP_WITHDRAWAL_QUOTA_FLOOR, which is permitted because the attacker's own weighted quota is also the floor: at a 10 USDC minimum, floor(1_000_000_000_000 * 10_000_001 / 500_000_000_000_000) = 20_000, raised to LP_WITHDRAWAL_QUOTA_FLOOR = 1_000_000. Then call assign_lp_quota_basis(amount = quota_basis_shares) with the victim's position as to_position. Since amount == basis_before, the carry is 100% of the spend.

The victim's next withdraw reverts WithdrawalExceedsLpHourlyQuota for the rest of the UTC hour and WithdrawalExceedsLpDailyQuota until 00:00 UTC. The assignment also zeroes the attacker's own counters, so their per-LP quota places no bound on how many times the cycle can run.

The victim has no on-chain escape. Rotating to a fresh wallet either leaves the basis behind, which weights the new position to zero, or carries it across with 100% of the

injected usage attached. close_lp_position requires settled_shares == 0, which only a withdraw clears. No ConfigParam variant writes these counters, and ADR004_HOURLY_LIMIT, ADR004_DAILY_LIMIT and LP_WITHDRAWAL_QUOTA_FLOOR are compile-time constants.

Impact

An unprivileged attacker can hold a chosen retail LP at zero withdrawal capacity for the remainder of each UTC window and renew that indefinitely. Neither the victim nor an operator, guardian or governance has a lever to clear the counters, and after deploy the only remedy is a program upgrade. Any zero-basis position sits at LP_WITHDRAWAL_QUOTA_FLOOR and is closed out by one cycle per day at any size; a $500,000 LP in a $500,000,000 pool takes 200 cycles to close the current hour and 1,001 to close the whole day, each cycle costing three instructions plus min_deposit - $1 stranded in basis-less shares. No funds are lost or confiscated: usage is conserved between the two positions, the aggregate ADR-004 ceiling is unchanged, institutional capital is out of reach because both legs are pinned to TRANCHE_RETAIL, and the denial lapses at each window boundary and has to be re-bought.

Recommendation

Require the receiver to consent, rederiving the seeds off the new signer:

```
pub to_owner: Signer<'info>,
#[account(
    mut,
    seeds = [LP_POSITION_SEED, to_owner.key().as_ref(), &[TRANCHE_RETAIL]],
    bump = to_position.bump,
    constraint = to_position.owner == to_owner.key()
        @ HousePoolError::InvalidLpPosition,
    constraint = to_position.tranche == TRANCHE_RETAIL
        @ HousePoolError::InvalidLpPosition,
    constraint = to_position.key() != from_position.key()
        @ HousePoolError::InvalidLpQuotaBasisAssignment,
)]
pub to_position: Box<Account<'info, LpPosition>>,
```

The buyer and seller the instruction exists to serve co-sign anyway, so this does not restrict its intended use.

Do not substitute a check that the assignment must increase the receiver's `quota_weight`. A zero-basis receiver goes from `min(live, 0) = 0` to `min(live, 1) = 1`, which satisfies that check, and zero-basis positions are the most exposed population here. If a second signature is not acceptable, use a two-step offer and accept, so that the consent still comes from the receiver.

Fix before deploy. There is no post-launch mitigation short of a program upgrade.

Three smaller items in the same instruction should be handled alongside it. The module header reasons only about sender-side inflation and forgery and says nothing about receiver consent, so update it once the fix lands. No test exercises the handler itself, so add a negative control asserting that a third party cannot alter another position's quota counters. The shipped IDL is missing this instruction along with eleven others and should be regenerated.

Status

Resolved

Low

[L-01] house-pool::cancel_institutional_redemption - Redemption request cancel-and-refile allows notice-window optionality

Description

An institutional LP may `request_institutional_redemption`, observe pool NAV movement during the 7-day notice window, then `cancel_institutional_redemption` and re-submit at will. No cool-down, no queue-position penalty, and no rate-limit on the cancel/refile cycle. This gives an institutional redeemer a free option: exit whenever NAV moves in their favor, reset whenever it moves against them.

Vulnerability Details

The `RedemptionRequest` account is closed on cancel with rent returned to the redeemer, and no per-LP timestamp is written back to `LpPosition` before the account vanishes. Because the handler body performs no rate-limit accounting either, the requester can immediately re-invoke `request_institutional_redemption` and start a fresh notice window with no memory of the prior request:

```
// programs/house-pool/src/instructions/cancel_institutional_redemption.rs:55-65
#[account(
    mut,
    close = redeemer,
    has_one = redeemer @ HousePoolError::UnauthorizedRedeemer,
    // ... no last_cancel_at write on LpPosition ...
)]
pub redemption_request: Box<Account<'info, RedemptionRequest>>,
```

The handler body itself is equally silent on cool-down bookkeeping:

```
// programs/house-pool/src/instructions/cancel_institutional_redemption.rs:68 onwards
pub fn handler(ctx: Context<CancelInstitutionalRedemption>) -> Result<()> {
    // ... emits CancelInstitutionalRedemptionEvent, returns Ok(()) ...
    // no last_redemption_cancel_at update, no cool-down bookkeeping
}
```

Impact

Institutional LPs have an asymmetric exit option relative to retail LPs and to the pool. Under adverse NAV motion within a notice window the LP cancels and re-arms rather than realizing the loss, shifting price risk onto retail holders and the house. This is not fund-loss but is a distribution unfairness.

Recommendation

Enforce a per-LP cool-down between cancellation and the next request, or make cancellation forfeit queue-priority:

```
require!(
    clock.unix_timestamp
        .saturating_sub(lp_position.last_redemption_cancel_at)
        >= REDEMPTION_CANCEL_COOLDOWN_SECONDS,
    HousePoolError::RedemptionCancelCooldownActive
);
```

Status

Resolved

[L-02] house-pool::authorize_game - Missing pool_state == Active gate allows game onboarding during pause

Description

The `authorize_game` handler mutates the game registry (`pool_config.authorized_game_count`, creates a `GameExposure` PDA) without gating on `pool_state`. Every fund-moving handler (`reserve_payout`, `collect_bet`, `release_payout`, `release_reserved`, etc.) correctly requires `pool_state == PoolState::Active`, so no bets can settle while paused - but games can still be onboarded, creating a state-machine inconsistency between the registry and the actual operational status.

Vulnerability Details

The `authorize_game::handler` checks only `operational_authority` and the game-registry invariants (unique program-id, count under `MAX_GAMES`), and does not read `pool_config.pool_state` or `emergency_pause_at`. In either `CircuitBreakerPaused` or `WindingDown` the handler still executes to completion, allowing new games to be silently onboarded into a non-operational pool. The account context makes the omission visible by contrast with the deposit/withdraw contexts elsewhere in the codebase - no `PoolState` check appears anywhere in the constraint list:

```
// programs/house-pool/src/instructions/authorize_game.rs (account struct excerpt)
#[account(
    mut,
    seeds = [POOL_CONFIG_SEED],
    bump = pool_config.bump,
    constraint = pool_config.operational_authority == authority.key()
        @ HousePoolError::UnauthorizedSigner,
    // no `pool_config.pool_state == PoolState::Active` gate here
)]
pub pool_config: Box<Account<'info, PoolConfig>>,
```

Impact

No fund-loss vector, since downstream fund-moving handlers block via their own `pool_state` gates. Impact is limited to governance-hygiene: (a) a paused pool can

accumulate authorized games that will go live the moment the pool is unpaused, and (b) audit-trail confusion during incident review.

Recommendation

Add a state gate consistent with the rest of the codebase:

```
require!(
    pool_config.pool_state == PoolState::Active,
    HousePoolError::PoolNotActive
);
require!(
    pool_config.emergency_pause_at == 0,
    HousePoolError::EmergencyPauseActive
);
```

Status

Resolved

[L-03] house-pool::recapitalize / house-pool::sweep_revenue - Recap-then-sweep sequence can suppress the insurance-draw trigger

Description

`recapitalize::handler` raises `pool_config.high_water_mark = max(current_high_water_mark, new_vault_balance)`. `sweep_revenue::handler` can then remove USDC from the vault. The combination lets an operational authority elevate HWM (which drives the insurance-trigger threshold, computed as `initial_pool_balance * insurance_trigger_ratio_bps`) and then reduce the vault, producing a state in which the vault sits below the trigger threshold yet is *not* eligible for a draw until it is re-baselined.

Vulnerability Details

Insurance-trigger evaluation in `math/circuit_breaker.rs:83-89` computes the threshold from `initial_pool_balance`, which is what `recapitalize` inflates, rather than from a moving reference tracked at sweep time. A recap-then-sweep cycle therefore keeps the trigger reference inflated while the actual `pool.pool_balance` is depressed, delaying legitimate insurance draws until a fresh deposit resets the baseline:

```
// programs/house-pool/src/math/circuit_breaker.rs:83-89
let trigger_balance = (pool.initial_pool_balance as u128)
    .checked_mul(config.insurance_trigger_ratio_bps as u128)
    .unwrap_or(0)
    / (BPS_SCALE as u128);
if pool.pool_balance < trigger_balance && pool.insurance_reserve == 0 { ... }
```

Impact

Requires the operational authority key (trusted role) to execute - this is not an unprivileged attack. Impact is limited to insurance-draw timing/observability; there is no direct fund loss. Emphasized here because the `PoolRecapitalized` event emitted by `recapitalize` does not record the authority class (only the funder), making incident review harder.

Recommendation

Either (a) require the recap authority to also update the trigger baseline coherently, or (b) log the class of authority used on `recapitalize` so post-hoc analysis can distinguish operational vs. treasury-initiated recaps. Additionally, monitor the (`recapitalize` → `sweep_revenue`) sequence off-chain.

Status

Resolved

[L-04] house-pool::deauthorize_game - GameExposure PDA never closed, rent permanently trapped and same program key can never be re-authorized

Description

The `deauthorize_game` handler sets `game_exposure.authorized = false` and decrements the game counters, but does not close the `GameExposure` PDA. The account persists on-chain forever with its rent locked, and - because the PDA seed is `[GAME_EXPOSURE_SEED, game_program.key()]` and `authorize_game` uses Anchor `init` (not `init_if_needed`) - the same `game_program` key can never be re-admitted to the registry. The operator is forced to redeploy any fixed game program under a new key to re-enable it, breaking clients that hardcode the program ID.

Vulnerability Details

In `deauthorize_game.rs:56-70`, the `GameExposure` account is declared with `mut` and constraints on `current_exposure == 0` and `authorized`, but no `close =` constraint is present, and the handler body only flips the `authorized` flag rather than closing the account:

```
// programs/house-pool/src/instructions/deauthorize_game.rs:56-70
#[account(
    mut,
    seeds = [GAME_EXPOSURE_SEED, game_program.key().as_ref()],
    bump = game_exposure.bump,
    constraint = game_exposure.current_exposure == 0
        @ HousePoolError::GameHasActiveExposure,
    constraint = game_exposure.authorized
        @ HousePoolError::GameNotAuthorizedForDeauth,
)]
pub game_exposure: Box<Account<'info, GameExposure>>,

// deauthorize_game.rs:93 - handler body only mutates the flag
exposure.authorized = false;
```

The design comment on the file (`deauthorize_game.rs:2-4`) explicitly states "does not close it; lifetime stats are preserved" - the retention of `total_bets_count`, `total_bets_volume`, `total_payouts`, and `total_collected` is the stated rationale.

However, these lifetime stats are already emitted in the `GameDeauthorized` event at handler exit (`deauthorize_game.rs:107-115`), so retaining them on-chain adds no observability that is not already available off-chain via indexers.

Because `authorize_game.rs:70-77` uses Anchor `init` (not `init_if_needed`) on the same PDA seed, a subsequent `authorize_game` call with the same `game_program` key deterministically fails at account creation time - Anchor rejects `init` when the account already exists.

Additionally, the ~0.00248 SOL rent paid by the operational authority at `authorize_game` is permanently locked in the deauth'd PDA with no on-chain recovery path.

Impact

A deauthorized game program cannot be re-authorized under the same program ID (Anchor `init` rejects the pre-existing PDA), forcing the operator to redeploy under a fresh key and orphan any off-chain state keyed by the old program ID. Approximately 0.00248 SOL (~$0.50) of rent per deauth is also permanently locked with no on-chain recovery path. Scoped to Low because `deauthorize_game` requires the operational authority (trusted role) and no direct USDC loss occurs; the stated design trade-off (preserve stats vs. permit re-authorization) is redundant because the stats are already event-emitted.

Recommendation

Either (a) close the PDA on deauth so `authorize_game` can re-init a fresh one:

```
#[account(
    mut,
    close = authority,
    seeds = [GAME_EXPOSURE_SEED, game_program.key().as_ref()],
    bump = game_exposure.bump,
    // ... existing constraints
)]
pub game_exposure: Box<Account<'info, GameExposure>>,
```

or (b) add a dedicated `re_authorize_game` handler that flips `authorized = true` back on an existing deauth'd PDA under the same signer/state constraints as `authorize_game`, guarded for idempotency:

```
require!(!exposure.authorized, HousePoolError::GameAlreadyAuthorized);
exposure.authorized = true;
pool_config.authorized_game_count = pool_config
    .authorized_game_count
    .checked_add(1)
    .ok_or(HousePoolError::ArithmeticOverflow)?;
```

Option (a) is the cleaner fix (rent recovered, lifecycle symmetric with the Reservation/escrow pattern used elsewhere). Option (b) preserves the on-chain lifetime stats if that is a hard requirement.

Status

Resolved

[L-05] transfer-hook::initialize_holder_status - Permissionless holder-status PDA initialization enables rent-DoS pre-init grief

Description

`initialize_holder_status` is permissionless: any caller can create a `HolderStatusV1` PDA for any target wallet with `status_flags = 0` (the safe default). Because the PDA is seeded on the target wallet's key (`[b"holder_status_v1", wallet.key().as_ref()]`) and Anchor `init` rejects a re-initialization attempt with `HolderStatusAlreadyInitialized` (error 7012), an attacker who pre-inits a victim's PDA blocks the victim from ever calling `initialize_holder_status` themselves. Compliance flags remain protected (only the bound Squads multisig can flip sanctioned/frozen bits via `set_holder_status`), so this is not a compliance bypass - but it is a footgun that lets an attacker force the victim to route around a squatted PDA and lose a small amount of rent to permanent grief.

Vulnerability Details

The `initialize_holder_status` handler contains no signer check on the target wallet - the caller is only required to pay rent (approximately 0.0015 SOL per PDA), and the wallet whose status is being initialized does not need to sign, leaving the target no on-chain veto. Aggregate grief cost across many targets scales linearly with rent, but each individual grief remains cheap enough to be nuisance-viable:

```
// programs/transfer-hook/src/instructions/initialize_holder_status.rs:59-98
pub fn initialize_holder_status_handler(ctx: Context<InitializeHolderStatus>) ->
Result<()> {
    let hs = &mut ctx.accounts.holder_status.load_init()?;
    hs.discriminator = holder_status_discriminator();
    hs.wallet = ctx.accounts.wallet.key().to_bytes();
    hs.status_flags = 0;  // safe default, cannot inject sanctioned/frozen
    // ... zero-initialise remaining fields ...
}
```

Note: sanctioned / frozen bits are gated behind `set_holder_status`, which requires the Squads multisig (error 7016 UnauthorizedSanctionsAuthority) - so a pre-init cannot inject harmful flags. The grief is limited to (a) forcing a targeted wallet to fund a workaround if they later need self-service PDA setup, and (b) minor rent-lock across many pre-inited PDAs.

Impact

No compliance bypass because sanctioned / frozen flags stay Squads-gated and a pre-init cannot forge them. The realistic damage is nuisance-tier: an attacker can spend a few thousand lamports each to pre-occupy any wallet's holder-status PDA, forcing that wallet to route around the squatted PDA in future flows and leaving small amounts of rent trapped across every griefed target.

Recommendation

Either (a) require the target wallet to co-sign initialization (add `wallet: Signer<'info>` to the account struct), OR (b) leave the current permissionless design and document the intended UX pattern (e.g., the platform UI always front-runs user-invoked init, so a squat only forces one extra step). Option (a) closes the grief window definitively; option (b) accepts the footgun in exchange for permissionless UX flexibility.

Status

Resolved

[L-06] game-router-ref::refund_stall - Revealed losses can be refunded after the full deadline

Description

Once a CommitReveal epoch seed has been revealed, every round in that epoch has a publicly verifiable outcome and should become settle-only. However, an Open losing round becomes refundable after the one-hour full deadline even though its loss is already known.

Vulnerability Details

The refund gate computes `full_window_elapsed` independently of the epoch's reveal state. Although the early-refund branch requires the seed to remain unrevealed, the final condition accepts either branch in `programs/game-router-ref/src/lib.rs:766-798`. Therefore, once `refund_deadline` elapses, a revealed losing round passes the gate solely because it is still Open.

The handler then calls `void_bet`, which returns the entire wager to the player's vault and unwinds the reservation, before marking the round Refunded in `programs/game-router-ref/src/lib.rs:800-839`. A later settlement cannot restore the valid loss because `settle_outcome` requires the round to remain Open in `programs/game-router-ref/src/lib.rs:466-480`.

The likelihood is low because the revealed loss must remain unsettled for the full 3,600-second deadline. CommitReveal settlement is permissionless, so the game backend, house keeper, or any other caller has an hour to settle the round. Nevertheless, the player has no economic reason to settle their own loss and can profit by waiting for the refund window whenever those other actors fail to act.

Proof of Concept

1. Place a CommitReveal wager whose deterministic payout will be zero.
2. Reveal the round's epoch seed and confirm that the round remains Open and the derived payout is zero.
3. Do not call `settle_outcome`; advance the Clock beyond REFUND_STALL_SECONDS,

which is 3,600 seconds in `programs/game-router-ref/src/state.rs:127-131`.

4. Call `refund_stall` with the correct revealed-epoch PDA.

5. Observe that the player's full wager is restored, the Reservation is closed, and the round becomes Refunded.

6. Attempt to settle the same round and observe that the Open-status guard rejects it.

Impact

The house pool does not receive the wager from a valid losing round. Instead, the player recovers the full stake and can retain the favorable side of the game while escaping any revealed loss that remains unprocessed for one hour.

Recommendation

For CommitReveal rounds, permanently disable refund as soon as the round's epoch seed has been revealed. A revealed Open round should remain settle-only regardless of elapsed time. Preserve liveness through a permissionless, preferably incentivized, resolution path that deterministically settles and releases the publicly derivable outcome rather than returning the wager.

Status

Resolved

[L-07] game-router-ref::initialize - Oracle keys cannot be rotated or revoked

Description

GameRouterConfig records an administrator, oracle authority, settlement source, and pause flag, and its documentation states that the administrator rotates the oracle and source in `programs/game-router-core/src/state/config.rs:33-47`. However, the reference integration writes these fields only during initialize in `programs/game-router-ref/src/lib.rs:76-104` and exposes no instruction that rotates or revokes the oracle, changes the source, or pauses settlement.

Vulnerability Details

The Ed25519 settlement path always verifies an attestation against the original router_config.oracle_authority in `programs/game-router-ref/src/lib.rs:567-588`. If that key is compromised, incident responders cannot invalidate it through the router's existing instruction surface. The compromised key remains able to authorize any claimed payout up to the round's reserved ceiling until the program is upgraded or settlement is contained outside the router.

If the key is merely lost, accepted oracle rounds cannot settle, but they are not permanently trapped: after the full deadline, `refund_stall` returns the wager and unwinds its reservation in `programs/game-router-ref/src/lib.rs:756-831`. The availability effect is therefore recoverable, while the compromised-key variant can affect repeated wagers.

The maximum financial impact is High and the likelihood of reaching it is Low because it requires compromise of the configured oracle. The oracle is intentionally trusted to attest outcomes for the settlement subsystem, so malicious use or compromise of that authority is subject to the trusted-control-plane cap. The resulting severity is Low.

Proof of Concept

1. Initialize a game with `SettlementSource::Ed25519Oracle` and oracle key K1.
2. Place a valid wager and reserve its maximum payout.
3. Use K1 to sign the canonical settlement digest with `claimed_payout` = reserved;

settlement and release accept the payout.

4. Attempt to replace or revoke K1 through the router program. No such instruction exists, so the persisted authority remains K1 and the same compromised key can continue authorizing subsequent open rounds.

Impact

A compromised oracle key can continue authorizing incorrect payouts for repeated wagers until an external containment action or program upgrade takes effect. A lost key temporarily prevents oracle rounds from settling and leaves wagers and reserved liquidity unavailable until their refund deadlines.

Recommendation

Implement an administrator-gated, versioned oracle lifecycle. Normal rotation should apply the new key only to newly accepted rounds, while each existing round remains bound to its accepted oracle and source version. Emergency revocation should immediately stop new wagers and reject further attestations from the compromised key, while preserving a reachable refund or alternative safe settlement path for every open round. Add a pause mechanism that blocks new exposure without disabling recovery or refunds.

Status

Resolved

[L-08] game-router-ref::refund_stall - Oracle rounds become early-refundable after five minutes

Description

`refund_stall` applies the CommitReveal-specific seed-reveal timeout to every settlement source. An Ed25519Oracle round whose oracle result has not settled becomes refundable after five minutes merely because it has no RevealedEpochSeed account, even though seed revelation is not the proof mechanism for that source.

Vulnerability Details

The oracle branch verifies a signed payout and does not consume the per-epoch seed account in `programs/game-router-ref/src/lib.rs:567-588`. Nevertheless, every round receives REVEAL_DEADLINE_SECONDS = 300 in `programs/game-router-ref/src/state.rs:133-145`.

After that deadline, `refund_stall` treats an absent seed PDA as an unrevealed CommitReveal epoch without checking router_config.settlement_source. Its `early_refund_unrevealed` condition therefore becomes true for an open oracle round in `programs/game-router-ref/src/lib.rs:756-798`. The handler then invokes `void_bet`, which returns the escrowed wager and unwinds the reservation before marking the round Refunded in `programs/game-router-ref/src/lib.rs:800-839`. The house-pool's shorter void timeout has already elapsed by this point in `programs/house-pool/src/instructions/void_bet.rs:158-190`.

The financially relevant variant requires the player to learn a losing oracle result while the round remains open, for example because an API exposes the result or a signed settlement transaction fails. The player can then choose the refund instead of accepting the loss. This prerequisite is not established by the on-chain code, so likelihood is Low. The loss is bounded to the affected wager or an unspecified set of similarly delayed rounds, giving Medium impact and Low severity.

Proof of Concept

1. Initialize a game with `SettlementSource::Ed25519Oracle`, create its required commitment, and place a valid wager.
2. Do not rotate the commitment, initialize the round's RevealedEpochSeed, or settle the oracle outcome.
3. Advance the validator clock by more than 300 seconds from the round's opening time.
4. Call `refund_stall` with the seeds-pinned but uninitialized revealed-seed PDA.
5. Observe that the full wager returns to the player, the Reservation closes, and the round becomes Refunded after only five minutes.
6. Attempt to settle the oracle outcome and observe that the Open status guard rejects it.

To demonstrate the economic variant, expose an off-chain oracle result with payout = 0 before step 3 and show that the player can recover the wager instead of accepting the known loss.

Impact

A player who learns a losing oracle result before settlement can recover the full wager after five minutes instead of realizing the loss. Even without result visibility, ordinary oracle delays prematurely cancel open wagers and unwind their reserved exposure under a timeout intended for unavailable CommitReveal seeds.

Recommendation

Restrict `early_refund_unrevealed` to `SettlementSource::CommitReveal`. Give Ed25519Oracle rounds a source-specific recovery deadline that does not depend on RevealedEpochSeed, or allow them to use only the full refund deadline. Apply oracle results atomically with settlement and do not rely on result confidentiality as the primary defense.

Status

Resolved

[L-09] revenue-distributor::initialize / claim_bucket - LP-yield bucket destination is not pinned at genesis, unlike the burn and insurance buckets

Description

`claim_bucket` requires the stored `lp_yield_wallet` to equal the derived house-pool `pool_vault` PDA, but `initialize` stores `lp_yield_wallet` without validating it against that PDA - unlike the burn and insurance buckets, which are both pinned at genesis. A genesis misconfiguration is therefore accepted silently and permanently disables all LP-yield (bucket 2) claims, with no setter to correct it in `programs/revenue-distributor/src/instructions/initialize.rs:231`.

Vulnerability Details

`initialize` re-derives and pins the burn and insurance destinations to their PDAs at genesis but stores `lp_yield_wallet` unchecked, while `claim_bucket` requires that same wallet to equal the house-pool `pool_vault` PDA:

```
// initialize.rs - burn & insurance are pinned at genesis
require!(config.burn_wallet     == buyback_usdc_vault,   ...); // :224-229
require!(config.insurance_vault == house_pool_insurance, ...); // :241-246
config.lp_yield_wallet = ctx.accounts.lp_yield_wallet.key();   // :231  <- stored,
NOT pinned

// claim_bucket.rs:143-154 - bucket 2 REQUIRES the pin genesis never enforced
require!(config.lp_yield_wallet == pool_vault, LpYieldWalletNotPoolVault);
// no instruction updates lp_yield_wallet after genesis -> a wrong value bricks
bucket-2 claims forever
```

Because no setter exists, a mismatched `lp_yield_wallet` silently passes `initialize` and then reverts every bucket-2 claim with `LpYieldWalletNotPoolVault`, stranding the largest bucket (2500 bps default) with no recovery short of an upgrade. The `claim_bucket` comment at `programs/revenue-distributor/src/instructions/claim_bucket.rs:139-142` inaccurately claims "the same two-gate pattern as the burn and insurance pins," but only the claim-time gate exists. Impact is high-if-triggered, yet the trigger requires an error by the build-pinned deploy authority

(`programs/revenue-distributor/src/instructions/initialize.rs:170-174`), is not attacker-reachable, and surfaces at the first claim, keeping it Low.

Impact

A single genesis error routes the LP-yield bucket to a value no claim can satisfy, permanently denying that bucket's share of every epoch's net profit until a program upgrade. No funds can be stolen - `claim_bucket` still enforces the pin - but the largest bucket's allocations stay locked with no post-genesis setter to correct it.

Recommendation

Add the genesis pin to `initialize`, mirroring the burn and insurance buckets. Immediately after `programs/revenue-distributor/src/instructions/initialize.rs:231`, derive the destination and assert equality:

```
let (pool_vault, _) = Pubkey::find_program_address(&[HOUSE_POOL_POOL_VAULT_SEED],
&HOUSE_POOL_PROGRAM_ID);
require!(config.lp_yield_wallet == pool_vault,
RevenueError::LpYieldWalletNotPoolVault);
```

This makes a misconfiguration fail at deploy time rather than silently bricking claims. Additionally, correct the "two-gate pattern" comment in `claim_bucket` to reflect the newly added genesis gate.

Status

Resolved

[L-10] staking-veswoobz::create_gpr_epoch / claim_gpr - Allocated GPR reward SWOOBZ has no reclaim, cancel, or withdrawal path and strands permanently in the reserve vault, unlike the LP-yield rail

Description

`create_gpr_epoch` permanently deducts `reward_pool_amount` from the `gpr_reserve_balance` accumulator when it allocates an epoch's rewards, but the SWOOBZ backing an unclaimed remainder - or an entire mispublished or never-activated epoch - can never be recovered. SWOOBZ leaves `gpr_reserve_vault` only through `claim_gpr` (valid proof, valid claimant, before the claim deadline); the program exposes no reclaim, cancel, or admin-withdraw instruction for the GPR rail, and no `ConfigParam` restores `gpr_reserve_balance`. Any SWOOBZ not claimed before the deadline is therefore stranded in the vault with no on-chain exit, in contrast to the sibling LP-yield rail, which frees unclaimed allocations via `reclaim_unclaimed_lp_yield` and `cancel_unpublished_lp_yield_epoch` in `programs/staking-veswoobz/src/instructions/create_gpr_epoch.rs:93-98`.

Vulnerability Details

The `gpr_reserve_balance` accumulator has exactly three writers and no restorer, and only `claim_gpr` moves SWOOBZ out of the vault:

```
// initialize.rs:143 - genesis
config.gpr_reserve_balance = 0;

// fund_gpr_reserve.rs:100-103 - grows by the post-fee SWOOBZ actually received (in)
config.gpr_reserve_balance += actual_received;

// create_gpr_epoch.rs:68-98 - allocation permanently DEDUCTS; never added back
require!(reward_pool_amount <= config.gpr_reserve_balance, RewardPoolExceedsReserve);
config.gpr_reserve_balance -= reward_pool_amount;

// claim_gpr.rs:180-226 - pays from the vault and bumps dist.total_claimed;
//                        does NOT restore gpr_reserve_balance
```

No instruction restores `gpr_reserve_balance`, and a repository search confirms `gpr_reserve_vault` is referenced only by `initialize_vaults` (create), `fund_gpr_reserve`

(in), and `claim_gpr` (out), while `update_staking_config`'s `ConfigParam` enum exposes no variant that writes `gpr_reserve_balance`. Two categories of SWOOBZ therefore strand permanently: the per-epoch unclaimed tail `reward_pool_amount - Σ(claims)` after `claim_deadline` (claimants who never claim leave a remainder that is neither re-allocatable, since the counter is not credited, nor withdrawable), and a mispublished or never-activated GPR root - because `publish_merkle_root` uses `init` the root is immutable and cannot be replaced, and with no cancel/reclaim the entire `reward_pool_amount` is locked.

The LP-yield rail handles exactly this: `reclaim_unclaimed_lp_yield` frees `allocated_usdc - total_claimed` back to the re-allocatable balance and `cancel_unpublished_lp_yield_epoch` frees a never-published epoch's whole allocation, keeping `lp_yield_outstanding` conserved. The GPR rail has no equivalent. Impact is bounded to the protocol's own reward capital with no attacker profit and no solvency break (the stranded SWOOBZ physically remains in the vault), and the full-pool case requires operator error, which is why it is scoped Low.

Impact

SWOOBZ allocated to GPR rewards but not claimed before the deadline is permanently inaccessible - it cannot be claimed, re-allocated, or withdrawn - so the protocol's usable reward capital is steadily reduced with no on-chain recovery short of a program upgrade. There is no fund-theft exposure or solvency break; the stranded SWOOBZ simply stays locked in `gpr_reserve_vault`.

Recommendation

Add a GPR recovery path mirroring the LP-yield rail. Two complementary pieces: (1) a `reclaim_unclaimed_gpr` instruction, authority-gated and callable only after `claim_deadline`, that credits `gpr_reserve_balance += reward_pool_amount - total_claimed` for the epoch, one-shot-guarded by a `reclaimed` flag on the distribution, so the remainder becomes re-allocatable; and (2) a `cancel_unpublished_gpr_epoch` for an epoch created but never published, freeing its `reward_pool_amount` back to `gpr_reserve_balance`. Alternatively, if unclaimed GPR rewards are intended to roll into the reserve, credit `gpr_reserve_balance` at reclaim so the physical SWOOBZ becomes

accounted and redeployable. Whichever path is chosen, preserve the solvency chain (`Σ reward_pool_amount <= funded`) and ensure it cannot double-free with `claim_gpr`, mirroring the `reclaimed` / `cancelled` one-shot guards the LP-yield rail already uses.

Status

Resolved

[L-11] buyback-burn::create_twap_order / execute_twap_slice - max_slippage_bps is read live per slice instead of pinned on the order, letting a mid-order governance change retroactively loosen an in-flight order's SWOOBZ floor

Description

`create_twap_order` snapshots the order's worst-acceptable acquisition price (`max_acquisition_price_micro`) onto the `TwapOrder` PDA, but the companion slippage parameter is not snapshotted: each slice's price-anchored floor is derived from `config.max_slippage_bps` read live at execution time. Because `max_slippage_bps` is independently mutable by the governance authority (a 48h-timelocked propose/execute), a slippage increase that lands while an order is in flight retroactively lowers the SWOOBZ floor for that order's remaining slices - an order the squads authority already priced under the old slippage in `programs/buyback-burn/src/instructions/execute_twap_slice.rs:161`.

Vulnerability Details

The order pins the price but the slice reads slippage live, so the two halves of the floor come from different points in time:

```
// create_twap_order.rs:172 - price is snapshotted onto the order
order.max_acquisition_price_micro = max_acquisition_price_micro;
// (there is no order.max_slippage_bps field)

// execute_twap_slice.rs:160-161 - price from the order, slippage read LIVE from
config
let max_acquisition_price_micro = order.max_acquisition_price_micro;
let max_slippage_bps = config.max_slippage_bps;          // <- NOT order-pinned
// :235 the floor uses both
let price_anchored_min =
    derive_min_swoobz_out(usdc_this_slice, max_acquisition_price_micro,
max_slippage_bps);
```

`max_slippage_bps` is bounded to `[MIN_SLIPPAGE_BPS, MAX_SLIPPAGE_BPS] = [10, 500]` and changed only through the governance-gated 48h timelock, so the floor can be loosened by at most ~4.9 percentage points of the ideal output, and only for slices

executed after the change lands. The value only actually leaves the protocol if the operational cranker is simultaneously delivering the minimum rather than its full bought balance, so this compounds the cranker-trust boundary rather than being independently triggerable by an outsider. It is nonetheless an avoidable inconsistency: an order's price is immutable once created, but the slippage that scales it is not, so the order's economic terms are not fully fixed at creation.

Impact

An in-flight order's per-slice SWOOBZ floor can be loosened after creation by a governance slippage increase, weakening the anti-under-delivery guarantee on its remaining slices. The effect is bounded by the 5% max-slippage cap and requires a cooperating cranker, so no outsider can trigger it.

Recommendation

Snapshot `max_slippage_bps` onto the `TwapOrder` at `create_twap_order` (mirroring `max_acquisition_price_micro`) and have `execute_twap_slice` derive the floor from the order-pinned value, so an order's economic terms are fixed at creation and cannot shift mid-flight. If live slippage is intentional, document that in-flight orders are subject to governance slippage changes and forbid raising `max_slippage_bps` while `active_order` is set.

Status

Resolved

[L-12] house-pool::close_player_vault - Permanently brickable by a one-micro-USDC token donation, nullifying the rent-reclaim remediation

Description

close_player_vault is one of the six close paths added to remediate M-04 (systemic PDA rent-lock). It gates on the player's segregated USDC vault holding a zero balance and performs no drain, so any party can permanently prevent the close by transferring one micro-USDC into the vault. The result is the same rent-lock M-04 was raised to fix.

Vulnerability Details

The handler requires the vault to be empty and then closes it:

```
require!(
    ctx.accounts.player_vault.amount == 0,
    HousePoolError::PlayerVaultNotEmpty
);
...
token::close_account(close_cpi)?;
```

The account struct separately requires player_balance.balance == 0 and player_balance.locked_in_play == 0. There is no path in the instruction that moves tokens out of player_vault before closing it, and SPL Token's own CloseAccount would reject a non-zero balance regardless.

player_vault is a PDA-derived token account whose address is publicly derivable from the player's public key, so anyone can send it dust. Once the accounting balance is zero and the token balance is not, the state is unrecoverable: player_withdraw debits player_balance.balance, which is already zero, so the donated surplus cannot be withdrawn, and the close can never succeed.

The escrow rail handles the same hazard correctly, where the fund-moving paths transfer the balance out and then close in the same instruction rather than requiring the account to already be empty.

The surplus condition was known. An in-struct comment notes that "A raw SPL transfer can create custody surplus that balance" does not reflect, and the response to it was a hard zero check rather than a drain.

Impact

The rent held by the PlayerBalance / PlayerVault pair is permanently locked for roughly one micro-USDC plus transaction fees per victim, which reproduces the M-04 impact through the instruction added to resolve it. The pool's own refund paths can also create the blocking state without any attacker, if uncredited escrow surplus is routed into a player vault. No user funds are at risk and no accounting invariant is broken; the loss is bounded by the rent-exempt minimum of the two accounts.

Recommendation

Make the handler drain-then-close rather than require-empty. Transfer any residual player_vault.amount to the appropriate destination under the vault PDA's signer seeds, then issue CloseAccount in the same instruction. Retain player_balance.balance == 0 and locked_in_play == 0 as the accounting-side gates, since those are the invariants that establish the vault is safe to close. Separately, confirm that void_bet, admin_void_bet and refund_unreserved_bet cannot route uncredited surplus into a player vault.

Status

Resolved

[L-13] transfer-hook::set_holder_status - Sanction target can revert an in-flight compliance action by reassigning one enumerated freeze target

Description

The M-06 remediation makes setting an enforcement flag atomic with freezing the sanctioned wallet's enumerated SWOOBZ token accounts. The freeze targets are validated before the flag is written and the whole instruction is atomic, so the subject of a pending sanction can force the transaction to fail by reassigning ownership of any one enumerated token account, obliging the compliance multisig to re-approve.

Vulnerability Details

The handler computes the updated flags under an immutable borrow, then validates and freezes, and only afterwards takes a mutable borrow to persist the flags:

```
validate_freeze_batch_shape(
    updated_flags,
    intended_target_count,
    ctx.remaining_accounts.len(),
)?;

if updated_flags & FLAGS_REQUIRING_NATIVE_FREEZE != 0 {
    validate_freeze_mint(...)?;
    validate_freeze_targets(
        ctx.remaining_accounts,
        &ctx.accounts.mint.key(),
        &ctx.accounts.wallet.key(),
    )?;
    freeze_unfrozen_targets(...)?;
}
```

validate_freeze_targets binds each supplied account to the sanctioned wallet and the canonical mint, which is what prevents an operator from freezing an unrelated victim's account. It also means the batch composition is fixed when the multisig proposal is built, while the on-chain state it is validated against stays under the subject's control until the transaction lands.

Token-2022 SetAuthority with AuthorityType::AccountOwner lets the subject reassign one of the enumerated accounts to a different owner. The account then fails validate_freeze_targets, the instruction returns an error, and Solana's atomicity reverts the flag write along with the freezes. The subject can repeat this for each new proposal. Squads multisig proposals are public on-chain state, so the pending action is observable in advance.

The all-or-nothing shape is deliberate and sound, since a partially applied sanction is worse than none, so this is a property of the design and not a defect in the guard. It is reported because the M-06 fix created it. At the audited baseline set_holder_status wrote the flag with no freeze batch, which gave the subject no revert lever.

This is distinct from the two M-06 operational behaviours already recorded in the retest (the MissingFreezeTargets fail-closed condition, and the absence of a thaw when flags are cleared).

Impact

An uncooperative sanction subject can delay a compliance action indefinitely at the cost of one SetAuthority transaction per attempt, forcing a full multisig re-approval round-trip each time. No enforcement is bypassed and no funds move; the flags either land completely or not at all. The impact is operational latency in a compliance workflow rather than a control failure.

Recommendation

Provide an operator procedure that closes the race rather than weakening the batch. The most direct is an atomic bundle that creates the token account set and lands the sanction in one transaction, leaving the subject no window in which to reassign.
An alternative is to make a single unfreezeable target non-fatal: skip targets that no longer belong to the subject, record the skipped set in the emitted event, and still commit the flags, so that hook-level enforcement lands even when the native freeze is incomplete. Whichever is chosen, record the in-flight invalidation behaviour in the compliance runbook alongside the two behaviours already documented.

Status

Acknowledged

[L-14] revenue-distributor::update_pool_tvl - Oracle update authority is write-once with no rotation path, and the module documentation asserts the opposite

Description

update_pool_tvl was added to resolve H-06, where profitable distribution reverted on a missing or stale TVL oracle. Its creation branch is gated on the governance authority and lets governance name any updater; every subsequent call is gated only on the key already stored in the account. The named key is therefore permanent for the life of the deployment. The module's own documentation states that the instruction rejects every updater except the governance authority, which holds only for the first call.

Vulnerability Details

The refresh branch checks the caller against the stored value and additionally forces the argument to match it:

```
require_keys_eq!(
    ctx.accounts.authority.key(),
    existing.update_authority,
    ...
);
require_keys_eq!(
    update_authority,
    existing.update_authority,
    ...
);
```

config.governance_authority is not an accepted signer on this branch. Enumerating the program's routed instructions shows no oracle-close, no oracle-authority rotation, and no handler that rewrites config.pool_tvl_oracle, which is assigned once during initialize and cannot be reassigned because the config is created with init. The oracle PDA is singleton-seeded, so no second oracle address is derivable, and distribute_epoch pins the account against distributor_config.pool_tvl_oracle.

Within its remaining scope the holder is unconstrained. tvl is an unbounded u64 written verbatim, with no sanity band and no cross-reference to the house-pool vault token

account, which is not present in the account context. Setting `tvl = 0` drives `insurance_cap` to zero in `apply_insurance_overflow`, redirecting the entire insurance allocation into the LP-yield bucket each epoch. Withholding updates past the staleness horizon stalls every profitable `distribute_epoch`.

The trade-off is worth stating. At the audited baseline there was no writer for this oracle at all, so the profitable-distribution branch was unconditionally dead, which is H-06 itself. The remediation strictly reduces the probability of that outage. It also introduces a non-governance, non-revocable principal on a money-path input where none previously existed, together with documentation that misdescribes the gate.

Impact

Loss or compromise of the named oracle key has no on-chain remedy after the upgrade authority is relinquished; recovery would require a program upgrade. Because the key is trusted and unconstrained, a faulty or hostile value skews the six-bucket revenue split, and withholding updates stalls distribution. All routes require the named signer, so this is not adversary-reachable by an unprivileged party. The documentation discrepancy is separately material, since a reviewer relying on the module comment would conclude the refresh path is governance-gated when it is not.

Recommendation

Accept `config.governance_authority` as an alternative signer on the refresh branch, and add an explicit rotation path for `PoolTvlOracle.update_authority`, before the upgrade authority is relinquished. Sanity-band tvl at the same time, preferably by reading the house-pool vault `TokenAccount` on-chain instead of trusting a pushed scalar, which removes the trusted-writer problem instead of managing it. Correct the module comment so it describes the two-branch gate accurately.

Status

Resolved

QA

[Q-01] house-pool::release_payout / house-pool::release_reserved - Wrong error variant on checked_sub failure

Description

`checked_sub` failures should surface as `HousePoolError::ArithmeticUnderflow`, but several call sites in the settlement handlers use `HousePoolError::ArithmeticOverflow` instead. Purely a debugging/observability nit - off-chain indexers keying on the error variant will misclassify the revert reason.

Vulnerability Details

The pattern shown below (from `release_payout.rs:435`) is repeated at several call sites in `release_payout.rs` and `release_reserved.rs:242-251`, all of which use the wrong error variant on a `checked_sub` failure. The correct variant is followed elsewhere in the codebase (e.g., `void_bet.rs:256` uses `ArithmeticUnderflow`), so this is a copy/paste inconsistency rather than a semantic disagreement:

```
// programs/house-pool/src/instructions/release_payout.rs:435
total_exposure = total_exposure
    .checked_sub(reserved_amount)
    .ok_or(HousePoolError::ArithmeticOverflow)?; // should be ArithmeticUnderflow
```

Impact

No fund-loss vector - the revert semantics are identical for both variants. Off-chain indexers and monitoring tools that key alerts on the error variant will misclassify the underflow revert as an overflow, obscuring root-cause diagnosis during a live incident.

Recommendation

Rename to the correct variant in both files:

```
.ok_or(HousePoolError::ArithmeticUnderflow)?;
```

Status

Resolved

[Q-02] house-pool::* (all fund-moving handlers) - sysvar_instructions validated imperatively instead of declaratively

Description

Every fund-moving instruction (`deposit`, `withdraw`, `deposit_institutional`, `request_institutional_redemption`, `execute_institutional_redemption`, `collect_bet`, `release_reserved`, `repay_backstop`, `top_up_tournament`, `fund_insurance`, `draw_insurance`, `recapitalize`, `emergency_pause`, `emergency_unpause`, `deactivate_circuit_breaker`, `void_bet`, `authorize_game`, `player_deposit`, `burn_bet_fee`, `reconcile_insurance`, `initialize_institutional_tranche`, `initialize_micro_burn_treasury`, `migrate_pool_config_v2`, and every other handler that calls `reject_durable_nonce` - 36 handlers in total across `programs/house-pool/src/instructions/`) declares its `sysvar_instructions` account as a bare `UncheckedAccount` and defers the pubkey validation to a `require_keys_eq!` inside `helpers::reject_durable_nonce`, called as the first statement of the handler body.

This is functionally safe today - the `require_keys_eq!` fully validates the account against the fixed Instructions sysvar id (`Sysvar1nstructions1111111111111111111111111`), and every current handler correctly calls the helper before any state mutation. The concern is purely defense-in-depth: a future contributor adding a new fund-moving instruction and copying the account-struct pattern without also copying the imperative check would introduce a durable-nonce replay window undetected by Anchor's declarative account validation.

Vulnerability Details

A representative declaration from `request_institutional_redemption.rs:39-40` shows the pattern: the account is declared as a bare `UncheckedAccount` with the CHECK doc-comment deferring validation to the helper:

```
// programs/house-pool/src/instructions/request_institutional_redemption.rs:39-40
/// CHECK: Sysvar verified inside reject_durable_nonce (replay reject).
pub sysvar_instructions: UncheckedAccount<'info>,
```

The actual key check lives in `helpers.rs:40-47`, invoked as the first statement of every fund-moving handler:

```
// programs/house-pool/src/helpers.rs:40-47
pub fn reject_durable_nonce(sysvar_instructions: &AccountInfo) -> Result<()> {
    require_keys_eq!(
        sysvar_instructions.key(),
        solana_program::sysvar::instructions::id(),
        HousePoolError::DurableNonceNotAllowed
    );
    // ...
}
```

The imperative-only pattern has two defense-in-depth consequences. First, validation only fires if the handler calls `reject_durable_nonce`; coverage today is complete (verified across all 36 declaration sites), but is enforced by author discipline rather than the type system, meaning any future contributor who forgets the imperative call introduces a silent replay window. Second, the check runs after all other account deserialization instead of at struct-parsing time, whereas Anchor's `#[account(address = ...)]` constraint would enforce the invariant at the account-parsing boundary before the handler body executes and would be visible to static-analysis tooling.

Impact

None today. The imperative check catches every attempted substitution, and every fund-moving handler is covered. The finding is filed as QA solely to harden the pattern against future handler additions.

Recommendation

Add a declarative address constraint to every `sysvar_instructions` account declaration so validation is enforced by Anchor's account-parsing pipeline in addition to the imperative check:

```
#[account(
    address = solana_program::sysvar::instructions::id()
        @ HousePoolError::DurableNonceNotAllowed
)]
```

```
/// CHECK: Validated by the address constraint above; content check in
reject_durable_nonce.
pub sysvar_instructions: UncheckedAccount<'info>,
```

Optionally, extend `reject_durable_nonce` coverage to `take_epoch_snapshot` - a stockpiled durable-nonce snapshot could be committed at a stale timestamp and confuse the epoch pipeline, though there is no direct fund-loss vector.

Status

Resolved

[Q-03] transfer-hook::build.rs - Missing all-or-none env-var binding guard (design inconsistency with swoobz-token/build.rs)

Description

`swoobz-token/build.rs:80-116` implements an ALL-OR-NONE guard that refuses to compile if any of the five build-bound pubkey pins is set without the others (deploy authority + mint + treasury + multisig + governance). This closes the partial-bind class where a missing env var would silently default to `[0u8;32]` (= System Program address for non-signer pins). `transfer-hook/build.rs` reads `SWOOBZ_MINT_PUBKEY` and `SWOOBZ_DEPLOY_AUTHORITY` independently with no equivalent guard. Verified as fail-safe today (both defaults are unusable pubkeys), but a future third pin would silently follow the same permissive default pattern unless the guard is added.

Vulnerability Details

In `swoobz-token/build.rs:74-116` an `is_pin_set` helper is combined with a filter over all pin env-vars, so the build panics with a legible diagnostic whenever some pins are set and others are not - closing the partial-bind failure mode by construction:

```
// programs/swoobz-token/build.rs:74-116
fn is_pin_set(env_var: &str) -> bool {
    env::var(env_var).map(|v| !v.trim().is_empty()).unwrap_or(false)
}

let set: Vec<&str> = PIN_ENV_VARS.into_iter().filter(|v| is_pin_set(v)).collect();
if !set.is_empty() && set.len() != PIN_ENV_VARS.len() {
    let unset: Vec<&str> = PIN_ENV_VARS
        .into_iter()
        .filter(|v| !is_pin_set(v))
        .collect();
    panic!("partial pin binding: set={set:?} unset={unset:?}");
}
```

The sibling `transfer-hook/build.rs` reads its two pins independently and never cross-references them - no `is_pin_set` helper and no partial-bind panic exists between the two `match` blocks, so a future third pin added on the same pattern would silently follow the permissive default:

```
// programs/transfer-hook/build.rs:29, 86
println!("cargo:rerun-if-env-changed=SWOOBZ_MINT_PUBKEY");
let bytes: [u8; 32] = match env::var("SWOOBZ_MINT_PUBKEY") { /* ... */ };
// ...
println!("cargo:rerun-if-env-changed=SWOOBZ_DEPLOY_AUTHORITY");
let deploy_bytes: [u8; 32] = match env::var("SWOOBZ_DEPLOY_AUTHORITY") { /* ... */ };
// no is_pin_set / partial-bind panic between the two matches
```

Impact

No exploitable issue today - both fail-closed defaults resolve to unusable pubkeys, so a partial binding still produces a non-functional program rather than a permissive one. The impact is a latent design inconsistency: adding a third pin in the future without also porting the all-or-none guard would silently reintroduce the partial-bind class the swoobz-token pattern explicitly closes.

Recommendation

Mirror the swoobz-token all-or-none pattern in `transfer-hook/build.rs` so any future added pin remains fail-secure by construction.

Status

Resolved

[Q-04] house-pool::update_immediate_parameters / house-pool::recapitalize - Account-name and event-audit-trail inconsistencies

Description

Two documentation/observability issues create ambiguity for downstream integrators. First, `update_immediate_parameters` binds an account named `governance` but constrains it against `operational_authority` - the naming implies a governance-class check while the actual binding is operational-class. Second, `recapitalize` accepts either `operational_authority` or `treasury_authority` but emits an event that records only `funder` and not which class authorized the call, which obscures the audit trail for compliance-relevant recap activity.

Vulnerability Details

The naming/constraint mismatch in `update_immediate_parameters.rs` binds a `Signer` labeled `governance` but the account constraint pins it to the `operational_authority` field of `pool_config` - a static reader following the parameter name would assume the guard is governance-class:

```
// programs/house-pool/src/instructions/update_immediate_parameters.rs:37-47
/// IMMEDIATE governance toggles. `None` fields are skipped.
/// operational class. Bound to `pool_config.operational_authority`.
pub governance: Signer<'info>, // ← named "governance" but bound to operational

#[account(
    mut,
    constraint = governance.key() == pool_config.operational_authority
        @ HousePoolError::UnauthorizedSigner,
)]
pub pool_config: Box<Account<'info, PoolConfig>>,
```

In parallel, the `recapitalize.rs` event omits the authority class entirely, so post-hoc analysis cannot tell whether operational or treasury signed:

```
// programs/house-pool/src/instructions/recapitalize.rs (event emit)
emit!(PoolRecapitalized {
    funder: ctx.accounts.funder.key(), // only funder recorded
```

```
    amount,
    // no field indicating whether operational or treasury authorized
});
```

Impact

No fund-loss vector - the underlying constraint is correctly enforced and both authority classes are legitimate signers for `recapitalize`. The impact is documentation clarity for downstream integrators reading the source and post-incident audit-trail resolution, where a compliance-relevant recap cannot be attributed to a specific authority class without off-chain correlation.

Recommendation

For the first issue, rename the account param to match its constraint (e.g., `operational` or `authority`). For the second, emit the authority class in the recap event:

```
#[event]
pub struct PoolRecapitalized {
    pub funder: Pubkey,
    pub authority_class: u8, // operational = 0, treasury = 1
    // ...
}
```

Status

Resolved

[Q-05] transfer-hook::execute - HolderStatusV1 zero-copy cast lacks discriminator check on the hot path

Description

The `PauseState` zero-copy cast in the hook checks its Anchor discriminator (`hash("account:PauseState")[..8]`) before trusting the bytes. The `HolderStatusV1` casts on both the source and destination legs do NOT perform the analogous discriminator check. The owner + PDA-address checks (primary anchors) currently make this safe, but a future addition of a same-seed `HolderStatusV2` struct or an accidental cross-struct write would silently be read as V1 layout on every transfer.

Vulnerability Details

The `PauseState` cast at `execute.rs:261-264` checks its Anchor discriminator before trusting the bytes, but the analogous check is missing from both the source and destination `HolderStatusV1` casts. The `PauseState` reference implementation looks like this:

```
// programs/transfer-hook/src/instructions/execute.rs:261-264
let expected_disc = solana_program::hash::hash(b"account:PauseState");
if ps.discriminator != expected_disc.to_bytes()[..8] {
    return err!(HookError::PausePdaMissing);
}
```

The source-side `HolderStatusV1` cast at `execute.rs:366-379` performs a length check but skips the discriminator comparison, so any account of the correct size and owner passes through:

```
// programs/transfer-hook/src/instructions/execute.rs:366-379
let src_data = source_holder_status.try_borrow_data()?;
if src_data.len() < std::mem::size_of::<HolderStatusV1>() {
    return err!(HookError::SourceUnverified);
}
let src: &HolderStatusV1 =
bytemuck::from_bytes(&src_data[..std::mem::size_of::<HolderStatusV1>()]);
source_tier = src.tier;
source_flags = src.status_flags;
```

The same omission repeats at `execute.rs:452-469` for the destination cast.

Impact

Currently safe because the owner check (account must be owned by the transfer-hook program) and the PDA-address re-derivation act as the primary anchors. The impact is a defense-in-depth gap: if a same-seed `HolderStatusV2` struct is later added or an accidental cross-struct write occurs, the hot path would silently read the old V1 layout without any typed rejection.

Recommendation

Add the same discriminator check pattern as `PauseState`:

```
let expected_disc = solana_program::hash::hash(b"account:HolderStatusV1");
if src.discriminator != expected_disc.to_bytes()[..8] {
    return err!(HookError::SourceUnverified);
}
```

Estimated CU cost: ~50 CU per cast. Well within the 10K CU hard ceiling (currently ~6,750 CU normal branch).

Status

Resolved

[Q-06] transfer-hook::initialize_extra_account_metas - Panic-via-.expect(...) in Anchor account-struct constant path

Description

The `space` parameter of the `extra_account_metas` init constraint uses `.expect(...)` on a runtime function call, meaning a failure would abort the transaction with a raw `ProgramFailedToComplete` panic rather than a typed `HookError`. `EXTRA_META_COUNT = 4` (compile-time constant) makes the panic unreachable in practice, but the pattern gives off-chain indexers no structured revert reason if the invariant ever breaks.

Vulnerability Details

The `space` parameter is computed inside the account-struct macro with a `.expect(...)` on a runtime function call, so any failure aborts the transaction with a raw `ProgramFailedToComplete` panic rather than a typed `HookError`. `EXTRA_META_COUNT = 4` is a compile-time constant so the panic is unreachable in practice, but the pattern gives off-chain indexers no structured revert reason if the invariant ever breaks:

```
// programs/transfer-hook/src/instructions/initialize_extra_account_metas.rs:30-36
#[account(
    init,
    payer = authority,
    space = ExtraAccountMetaList::size_of(EXTRA_META_COUNT)
        .expect("extra_account_meta space calculation failed"),
)]
```

Impact

Unreachable in practice - `EXTRA_META_COUNT` is a compile-time constant so the fallible size calculation never actually fails at runtime. Purely a code-hygiene concern: if the invariant were ever broken by a refactor, off-chain indexers would receive a raw `ProgramFailedToComplete` panic instead of a structured `HookError`, making the failure harder to attribute.

Recommendation

Replace with a `const fn` computation and a compile-time assertion, or move the space calculation into the handler body with a typed `HookError` mapping.

Status

Resolved

[Q-07] swoobz-token::initialize_mint - MetadataPointer and TokenMetadata update authorities set to fee_config_authority PDA but no update handler exists (implicit immutability by handler-absence)

Description

`initialize_mint` sets the update authority on both the `MetadataPointer` extension (line 495) and the `TokenMetadata` extension (line 530) to the program's `fee_config_authority` PDA. The `fee_config_authority` PDA can only be signed by this program via `invoke_signed`, and the program contains NO handler that constructs `update_metadata_pointer` or `update_field` instructions - so the metadata is effectively frozen at genesis. However, this immutability is enforced only by the ABSENCE of an update handler, not declaratively at the Token-2022 layer. A future refactor that adds a new handler which signs for the `fee_config_authority` PDA (for any purpose) could accidentally expose metadata mutation as a side effect if that handler constructs the wrong Token-2022 instruction.

Vulnerability Details

The `MetadataPointer` initialization at `initialize_mint.rs:487-500` sets the update authority to the program's `fee_config_authority` PDA, meaning any future handler that acquires a signature for that PDA (for any purpose) implicitly gains metadata-mutation power unless the corresponding Token-2022 instruction is deliberately withheld:

```
// programs/swoobz-token/src/instructions/initialize_mint.rs:487-500
let ix = spl_token_2022::extension::metadata_pointer::instruction::initialize(
    token_program_id,
    mint_key,
    Some(ctx.accounts.fee_config_authority.key()), // ← update authority
    Some(*mint_key),
)?;
```

The `TokenMetadata` initialization at `initialize_mint.rs:518-546` follows the same pattern for the metadata itself:

```
// programs/swoobz-token/src/instructions/initialize_mint.rs:518-546
let ix = spl_token_metadata_interface::instruction::initialize(
```

```
    token_program_id,
    mint_key,
    &ctx.accounts.fee_config_authority.key(), // ← update authority
);
```

Impact

Metadata immutability is preserved today because no handler in the program constructs `update_metadata_pointer` or `update_field` instructions, so the `fee_config_authority` PDA cannot reach either Token-2022 setter. A future refactor that adds any handler which signs for `fee_config_authority` for an unrelated purpose could accidentally expose metadata mutation as a side effect, silently converting the mint's supposedly-fixed name / symbol / URI into a mutable surface.

Recommendation

Explicitly set the update authority to `None` at initialization to make the immutability enforced at the Token-2022 layer:

- `MetadataPointer`: pass `None` instead of `Some(fee_config_authority.key())`.
- `TokenMetadata`: after init, call `spl_token_metadata_interface::instruction::update_authority` to set `new_authority = None` (irrevocable).

Alternatively, add a load-bearing code comment stating that immutability is invariant on the absence of an update handler.

Status

Resolved

[Q-08] staking-veswoobz::create_gpr_epoch - Admin-supplied epoch parameter unconditionally overwrites config.current_epoch, allowing non-monotonic epoch pointer that disrupts the vote window

Description

`create_gpr_epoch` accepts an `epoch: u64` instruction argument and, on success, does `config.current_epoch = epoch` without any monotonicity guard (no `epoch > config.current_epoch` check). Because `vote::handler` gates every vote on `epoch == staking_config.current_epoch`, any admin call that supplies an epoch value LOWER than the current pointer (or a large sparse gap forwards) silently redirects the "active" voting window. A single admin mistake (accidentally re-typing an old epoch number, or a UI bug passing a stale value) or a compromised admin key can move the vote window backwards to a stale epoch or forwards into an unfunded slot, disrupting reward accounting without a fund-loss vector. The `init` constraint on the `GprEpochState` PDA prevents re-initializing the same epoch number, but does not order them. Admin authority is a Squads multisig, so this is an admin-footgun rather than a permissionless attack, hence QA severity.

Vulnerability Details

The `create_gpr_epoch` handler accepts an `epoch: u64` instruction argument and writes it to `config.current_epoch` at the end of a successful call without any monotonicity guard, so `epoch` may be any value the admin supplies:

```
// programs/staking-veswoobz/src/instructions/create_gpr_epoch.rs:52-99
pub fn handler(
    ctx: Context<CreateGprEpoch>,
    epoch: u64,
    proposals_count: u8,
    reward_pool_amount: u64,
) -> Result<()> {
    // ... allocation checks ...
    config.current_epoch = epoch; // line 99: no monotonic guard
}
```

The downstream vote gate reads that same `current_epoch` field and blocks any vote against a different epoch, so the write above directly controls which epoch's vote window is currently open:

```
// programs/staking-veswoobz/src/instructions/vote.rs:94
require!(
    epoch == ctx.accounts.staking_config.current_epoch,
    StakingError::EpochMismatch,
);
```

The combined effect is that if admin calls `create_gpr_epoch(100, ...)` and then later `create_gpr_epoch(1, ...)`, `current_epoch` is set to `1` and all voting for epoch 100 is silently blocked mid-window even though epoch 100's `GprEpochState` PDA remains live on-chain.

Impact

No direct fund-loss vector - voting is a gate on reward eligibility rather than any transfer, and admin authority is a Squads multisig. The realistic damage is governance disruption: votes cast between two out-of-order admin calls are recorded under the wrong window, and off-chain merkle builders that assume monotonic epochs may skip or double-count rewards. Materialises only on a Squads mistake or compromise, hence QA severity.

Recommendation

Add a strict monotonicity guard at the top of `create_gpr_epoch::handler`:

```
require!(
    epoch > config.current_epoch,
    StakingError::EpochMustAdvance,
);
```

Alternatively, derive `epoch` INSIDE the handler as `config.current_epoch + 1` (dropping it from the instruction args entirely), which closes the footgun definitively while trivially preserving the intended one-epoch-forward semantic.

Status

Resolved

[Q-09] revenue-distributor::vote_emergency_buyback - deprecated no-op with a misleading "vote" name

Description

`vote_emergency_buyback` is a public instruction whose handler validates the caller and returns `Ok(())` without mutating any state, emitting any event, or granting any authority. It ignores its `_support: bool` argument and confers no on-chain approval. It is not dead (unreachable) code - it is reachable and succeeds - but it is an intentional no-op stub retained for client/indexer compatibility, as its own header documents in `programs/revenue-distributor/src/instructions/vote_emergency_buyback.rs:1-9`.

Vulnerability Details

The handler performs only replay protection and a Squads-signer check, then returns:

```
// vote_emergency_buyback.rs:45-58
pub fn handler(ctx: Context<VoteEmergencyBuyback>, _support: bool) -> Result<()> {
    reject_durable_nonce(&ctx.accounts.sysvar_instructions)?;
    require_keys_eq!(
        ctx.accounts.squads_multisig.key(),
        ctx.accounts.distributor_config.squads_multisig,
        RevenueError::UnauthorizedGovernance
    );
    Ok(()) // mutates nothing, emits nothing, grants nothing
}
```

There is no security impact: the endpoint is gated to `config.squads_multisig`, touches no state, and cannot bypass the emergency-buyback timelocks (real approval flows through Squads' m-of-n plus the propose/execute dual timelocks). The only concern is clarity: an instruction literally named `vote` records no vote, and because it ignores `_support` and emits nothing, an off-chain indexer cannot distinguish a "yes" from a "no" or reliably detect intent - so an integration could mistakenly treat a successful call as a recorded approval. The header already warns "New integrations should not call it," so the risk is limited to naming/semantics.

Recommendation

Remove the endpoint in the next ABI-breaking release to shrink the surface and eliminate the misleading name. If it must be retained for compatibility, have it emit an explicit deprecation event and document that a successful call confers no approval, so no off-chain system mistakes it for an on-chain vote.

Status

Resolved

[Q-10] buyback-burn::execute_twap_slice - Cranker token accounts lack explicit mint constraints, relying on implicit CPI-level enforcement

Description

In `execute_twap_slice`, the cranker's two token accounts - `cranker_usdc` (the USDC destination of Step 1) and `cranker_swoobz` (the SWOOBZ source of Step 2) - are declared as bare `mut` token accounts with no `token::mint` or `token::authority` constraint. Their correctness is enforced only implicitly: the legacy SPL `transfer` requires `from.mint == to.mint`, and the Token-2022 `transfer_checked` is passed the pinned `swoobz_mint`, so a wrong-mint account makes the CPI fail rather than mis-execute. The behaviour is safe today, but the guarantee is emergent from the CPI instead of stated at the account boundary, unlike the vault accounts which are seed-pinned.

Vulnerability Details

The cranker accounts carry no declarative mint (or authority) binding:

```
// execute_twap_slice.rs:101-107
/// Cranker's USDC token account (receives USDC to execute swap off-chain).
#[account(mut)]
pub cranker_usdc: Box<Account<'info, TokenAccount>>,

/// Cranker's SWOOBZ token account (deposits bought SWOOBZ into vault).
#[account(mut)]
pub cranker_swoobz: Box<InterfaceAccount<'info, TokenAccountInterface>>,
```

Both mints are already present in the account context (`swoobz_mint` directly; the USDC mint via the seed-pinned `usdc_vault`), so an explicit `token::mint = swoobz_mint` on `cranker_swoobz` (and an equivalent USDC binding) would move the check from the CPI layer to account validation. This is a defense-in-depth and readability improvement, not a live vulnerability - a mismatched mint fails the transfer and reverts the whole slice.

Recommendation

Add explicit `token::mint = ...` constraints to `cranker_usdc` and `cranker_swoobz` so the mint binding is declarative and fails fast at account validation, matching the constraint

style used elsewhere in the program (for example `create_twap_order`'s triple-bound `usdc_vault`). Optionally add `token::authority = cranker` to state the ownership requirement explicitly as well.

Status

Resolved

Centralisation

The Swoobz project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Swoobz project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au